

0_10xPBMC_SoloTE

August 31, 2026

```
[1]: library(Seurat)
library(ggplot2)
library(pheatmap)
library(dplyr)
library(RColorBrewer)
library(clustree)
library(reshape2)
library(ggpubr)
getwd()
dir.create("figures_10xPBMC")
dir.create("data_10xPBMC")
dataset_id <- "10xPBMC"
sample_id <- "pbmc8k"
colorPBMC <- "#81B29A"
```

Loading required package: SeuratObject

Loading required package: sp

Attaching package: 'SeuratObject'

The following objects are masked from 'package:base':

intersect, t

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

```
intersect, setdiff, setequal, union
```

Loading required package: ggraph

Attaching package: 'ggraph'

The following object is masked from 'package:sp':

```
geometry
```

```
~/mnt/TEdata/TEbenchmarking/jupyterNotebooks/realDatasets'
```

```
Warning message in dir.create("figures_10xPBMC"):
```

```
"'figures_10xPBMC' already exists"
```

```
Warning message in dir.create("data_10xPBMC"):
```

```
"'data_10xPBMC' already exists"
```

```
[2]: # celltype annotation

celltypes <- read.table("TEbenchmarking/data/whitelists/
  ↪celltype_annotation_sub_min.tsv") # same file passed to snakemake

celltypes$V2 <- gsub(pattern="CD14pos_Monocytes", replacement = "
  ↪Monocytes_CD14pos", celltypes$V2)
celltypes$V2 <- gsub(pattern="FCGR3Apos_Monocytes", replacement = "
  ↪Monocytes_FCGR3Apos", celltypes$V2)
celltypes$V2 <- gsub(pattern="Naive_CD4_T", replacement = "T_CD4_Naive",
  ↪celltypes$V2)
celltypes$V2 <- gsub(pattern="Memory_CD4_T", replacement = "T_CD4_Memory",
  ↪celltypes$V2)
celltypes$V2 <- gsub(pattern="CD8A", replacement = "T_CD8", celltypes$V2)
celltypes$V2 <- gsub(pattern="Megakaryocytes", replacement = "Platelet",
  ↪celltypes$V2)

PBMCelltypeColors <- c("B"="#91bcca",
  "Monocytes_CD14pos"="#3d405b",
  "T_CD8"="#f2cc8f",
  "Dendritic"="#c492a7",
  "Monocytes_FCGR3Apos"="#e78063",
  # "Platelet"="#c4b3ab",
  "T_CD4_Memory"="#81b29a",
  "T_CD4_Naive"="#4d7362",
```

```
"NK"= "#88535a")
```

```
[3]: getwd()

conversionTable <- read.table("annotation/annotation_hg38_conversion_withAge.
  ↪tsv")
head(conversionTable)
```

```
'/mnt/TEdata/TEbenchmarking/jupyterNotebooks/realDatasets'
```

A data.frame: 6 × 16

	chr	start	end	strand	sw_score	locusID	family	subfam
	<chr>	<int>	<int>	<chr>	<int>	<chr>	<chr>	<chr>
1	chr1	10469	11447	-	3612	TAR1	telo	TAR1
2	chr1	11505	11675	-	484	L1MC5a_dup3	L1	L1MC
3	chr1	11678	11780	-	239	MER5B_dup1	hAT-Charlie	MER5
4	chr1	15265	15355	-	318	MIR3_dup11	MIR	MIR3
5	chr1	18907	19048	+	239	L2a_dup29	L2	L2a
6	chr1	19972	20405	+	994	L3_dup3	CR1	L3

```
[4]: SoloTE_path <- paste0("/mnt/volume_1p5T/results/SoloTEout/", dataset_id, "/",
  ↪sample_id, "/", sample_id, "_SoloTE_output/", sample_id, "_legacytes_MATRIX")
STAR_path <- paste0("/mnt/TEresults/snake_make_results/results/STARoutdir/
  ↪", dataset_id, "/", sample_id, "/best_Solo.out/Gene")

legacyTEmatrix <- Seurat::ReadMtx(mtx = paste0(SoloTE_path, "/matrix.mtx"),
  cells = paste0(SoloTE_path, "/barcodes.tsv"),
  features = paste0(SoloTE_path, "/features.tsv"))
  ↪# read matrix

# select TEs
TEs <- grep("SoloTE", rownames(legacyTEmatrix), value = T)
locusTEs <- grep("chr", TEs, value = T)
# subset the matrix keeping only TEs
TEmatrix <- legacyTEmatrix[locusTEs,]
# remove "SoloTE" from the name of the TEs
rownames(TEmatrix) <- gsub("SoloTE\\|", "", rownames(TEmatrix))

rownames(TEmatrix) <- gsub("\\|", "-", rownames(TEmatrix))
rownames(TEmatrix) <- gsub("\\_", "-", rownames(TEmatrix))
rownames(TEmatrix) <- gsub("\\?", "", rownames(TEmatrix))

#table(rownames(TEmatrix) %in% conversionTable$soloteID)
# transform into Stellarscope IDs
#rownames(TEmatrix) <- conversionTable$stellarscopeID[match(rownames(TEmatrix),
  ↪conversionTable$soloteID)]
nCells <- ncol(TEmatrix)
thrMinCells <- round(nCells * 0.05)
```

```
# create Seurat object with shallow filtering of TEs expressed in at least 5%
↳ of cells and cells expressing at least 50 TEs
objTE <- Seurat::CreateSeuratObject(TEmatrix, project = "PBMC",
                                   min.cells = thrMinCells, min.features = 50)
objTE
```

An object of class Seurat
 6758 features across 1188 samples within 1 assay
 Active assay: RNA (6758 features, 0 variable features)
 1 layer present: counts

```
[9]: # select genes
genes <- setdiff(rownames(legacyTEmatrix), TEs)
# subset the matrix keeping only genes
Genematrix <- legacyTEmatrix[genes,]
# create Seurat object with shallow filtering of genes expressed in at least 50
↳ cells and cells expressing at least 100 genes
Gene <- Seurat::CreateSeuratObject(Genematrix, project = "PBMC",
                                   min.cells = thrMinCells, min.features = 100)

filteredBarcodes <- read.table(paste0(STAR_path, "/filtered/barcodes.tsv"))$V1
↳ # read barcodes selected by STARsolo

# barcodes of platelets
plateletBarcodes <- celltypes[celltypes$V2=="Platelet", "V1"]

# remove low quality cells and platelets
objTE_SoloTE <- objTE[, setdiff(filteredBarcodes, plateletBarcodes)] # keep only
↳ filtered barcodes
objGenes_SoloTE <- Gene[, setdiff(filteredBarcodes, plateletBarcodes)] # keep
↳ only filtered barcodes

objTE_SoloTE
objGenes_SoloTE
```

An object of class Seurat
 6758 features across 1056 samples within 1 assay
 Active assay: RNA (6758 features, 0 variable features)
 1 layer present: counts

An object of class Seurat
 7334 features across 1056 samples within 1 assay
 Active assay: RNA (7334 features, 0 variable features)
 1 layer present: counts

```
[10]: # QC TEs

options(repr.plot.width=7, repr.plot.height=6)

objTE_SoloTE@meta.data$nCount_TE <- objTE_SoloTE@meta.data$nCount_RNA
objTE_SoloTE@meta.data$nFeature_TE <- objTE_SoloTE@meta.data$nFeature_RNA

# Visualize QC metrics as a violin plot
VlnPlot(objTE_SoloTE, features = c("nCount_RNA", "nFeature_RNA"), ncol = 2,
        cols = colorPBM, pt.size = 0, alpha = 0.5)
summary(objTE_SoloTE$nCount_RNA)
summary(objTE_SoloTE$nFeature_RNA)
```

Warning message:

"Default search for "data" layer in "RNA" assay yielded no results; utilizing "counts" layer instead."

Warning message:

"The `slot` argument of `FetchData()` is deprecated as of SeuratObject 5.0.0.

Please use the `layer` argument instead.

The deprecated feature was likely used in the [Seurat](#) package.

Please report the issue at

[<https://github.com/satijalab/seurat/issues>.](https://github.com/satijalab/seurat/issues)"

Warning message:

"`PackageCheck()` was deprecated in SeuratObject 5.0.0.

Please use `rlang::check\_installed()` instead.

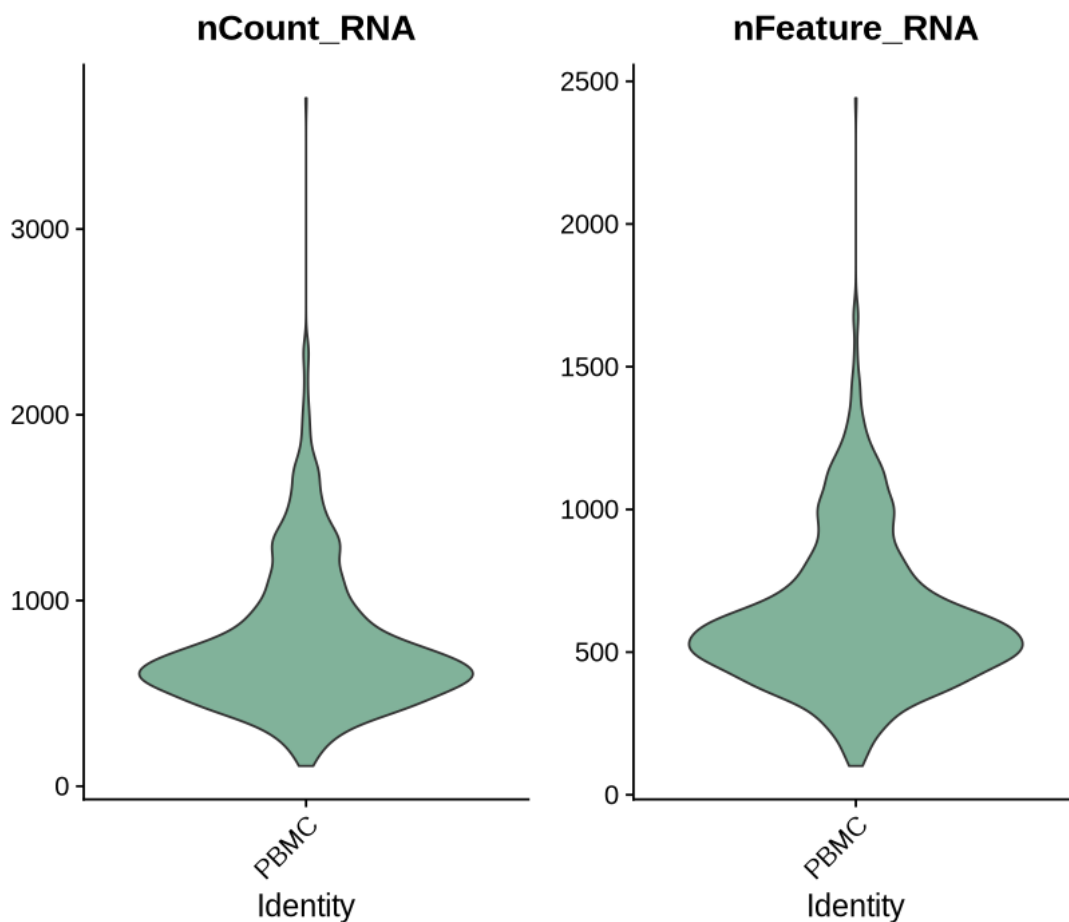
The deprecated feature was likely used in the [Seurat](#) package.

Please report the issue at

[<https://github.com/satijalab/seurat/issues>.](https://github.com/satijalab/seurat/issues)"

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
109.0	530.0	668.5	777.7	940.0	3705.0

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
101.0	462.0	573.0	629.4	747.0	2441.0



```
[11]: # QC genes
options(repr.plot.width=8, repr.plot.height=5)

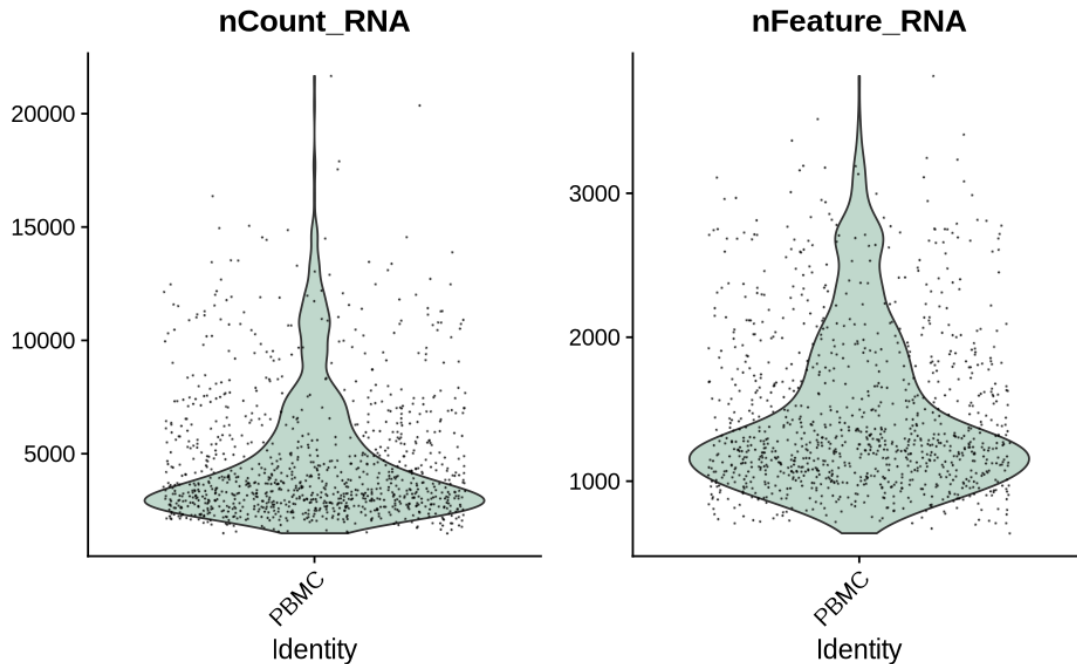
VlnPlot(objGenes_SoloTE, features = c( "nCount_RNA", "nFeature_RNA" ), ncol = 2,
        cols = alpha(colorPBMC, 0.5), pt.size = 0.05, alpha = 0.5)

summary(objGenes_SoloTE$nFeature_RNA)
summary(objGenes_SoloTE$nCount_RNA)
```

Warning message:

"Default search for "data" layer in "RNA" assay yielded no results; utilizing "counts" layer instead."

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
638	1068	1276	1462	1754	3814
Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1490	2808	3676	4714	5645	21658



1 Genes

```
[12]: objGenes_SoloTE <- NormalizeData(objGenes_SoloTE, normalization.method = "
      ↪LogNormalize", scale.factor = 10000)

objGenes_SoloTE <- FindVariableFeatures(objGenes_SoloTE, selection.method = "
      ↪vst", nfeatures = 4000)

# Identify the 10 most highly variable genes
top10 <- head(VariableFeatures(objGenes_SoloTE), 10)

# plot variable features with and without labels
plot1 <- VariableFeaturePlot(objGenes_SoloTE)
plot2 <- LabelPoints(plot = plot1, points = top10, repel = TRUE)
plot2
```

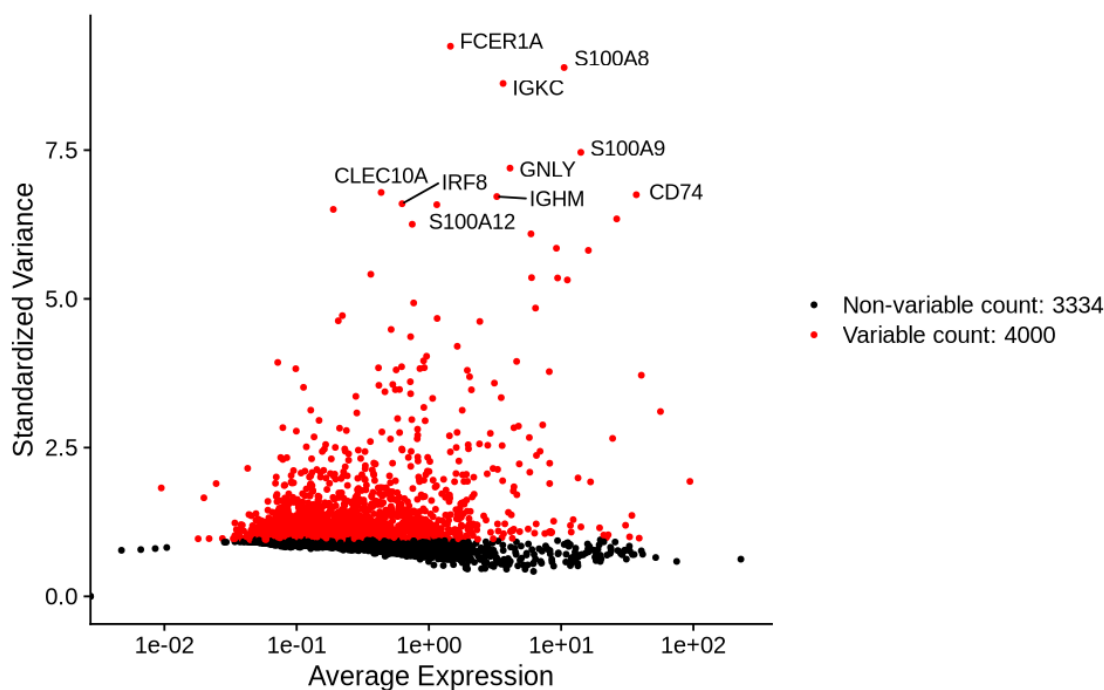
Normalizing layer: counts

Finding variable features for layer counts

When using repel, set xnuage and ynuage to 0 for optimal results

Warning message in scale_x_log10():

"log-10 transformation introduced infinite values."



```
[13]: gc()
all.genes <- rownames(objGenes_SoloTE)
objGenes_SoloTE <- ScaleData(objGenes_SoloTE) # on hvg

objGenes_SoloTE <- RunPCA(objGenes_SoloTE, features = VariableFeatures(object =
  objGenes_SoloTE))

DimPlot(objGenes_SoloTE, reduction = "pca") + NoLegend()

ElbowPlot(objGenes_SoloTE)
```

		used	(Mb)	gc trigger	(Mb)	max used	(Mb)
A matrix: 2 × 6 of type dbl	Ncells	27816780	1485.6	63935878	3414.6	42920484	2292.3
	Vcells	244964168	1869.0	423138372	3228.3	422858482	3226.2

Centering and scaling data matrix

PC_ 1

Positive: CD3E, TRAC, IL32, CD3D, TRBC2, LCK, CD69, PCED1B-AS1, CD7, AES
 IFITM1, IL7R, CD27, BCL11B, STK17A, CD3G, CLEC2D, ETS1, CTSW, CD247
 SPOCK2, CD2, RHOH, TRBC1, FCMR, GZMA, GZMM, CCL5, LDHB, KLRB1

Negative: CST3, LST1, AIF1, TYMP, SPI1, MNDA, CSTA, LYZ, CTSS, LGALS1
 S100A11, SERPINA1, FTH1, PSAP, FTL, KLF4, FCER1G, TYROBP, FCN1, COTL1

GRN, FGL2, NCF2, HLA-DRB1, NPC2, CPVL, AP1S2, SAT1, HCK, CLEC7A

PC_ 2

Positive: NKG7, PRF1, GZMA, KLRD1, CST7, FGFBP2, GNLY, GZMB, CTSW, KLRF1
SPON2, HOPX, CLIC3, TRDC, CD160, TTC38, MATK, KLRB1, CD247, IL2RB
PFN1, CD7, CMC1, GZMM, ADGRG1, FCGR3A, TBX21, ITGB2, CCL5, S1PR5

Negative: IGHM, IGHD, CD79A, RPS23, MS4A1, IGKC, RPLP1, RPS2, AL139020.1,
LINC00926

RPS18, BANK1, CD22, CD79B, FCER2, MEF2C, TNFRSF13C, HLA-DRA, VPREB3,
RALGPS2

BCL11A, FCRLA, EAF2, HLA-DQB1, CD19, FAM129C, PLPP5, LINC02397,
FCRL1, TSPAN13

PC_ 3

Positive: NAPSB, HLA-DPA1, HLA-DQA1, HLA-DPB1, CD74, FCER1A, HLA-DQB1, HLA-DMA,
BCL11A, PDLIM1

HLA-DMB, BANK1, CD79A, MS4A1, ADAM28, IGHM, BASP1, CD1C, IGHD, HLA-
DRA

IRF8, GSN, CD22, FLT3, ENHO, SPIB, CYB561A3, AL139020.1, BLNK, IGKC

Negative: CFD, SMIM25, S100A6, FTL, SERPINA1, CDA, DUSP6, C5AR1, RBP7, NFKBIZ
S100A4, NEAT1, CD14, S100A8, MS4A7, S100A12, MAFB, SLC11A1, FCN1,
S100A9

CD3D, AIF1, IFITM3, CD68, STXBP2, TRAC, CLEC7A, NINJ1, TCF7L2, NAMPT

PC_ 4

Positive: LDHB, VIM, RPS18, RPLP1, FCER1A, TRAC, CD3D, RGS10, RPS2, IL7R
CD3E, RPS24, CD3G, IL32, JAML, S100A10, SNHG29, ALDH2, PPA1, GPR183
CLEC10A, CD2, BCL11B, IL6R, PLP2, LEF1, CACNA2D3, ENHO, ACTN1, NOSIP

Negative: FCGR3A, CD79B, IGHM, CD79A, IGHD, MS4A1, PLAC8, AL139020.1, FCER2,
LINC00926

IGKC, RHOC, SMIM25, MTSS1, MS4A7, TNFRSF13C, HES4, VPREB3, TCF7L2,
CALHM6

BANK1, HVCN1, LYN, CD22, CDKN1C, PLPP5, FCRLA, CHST2, RALGPS2, FCRL1

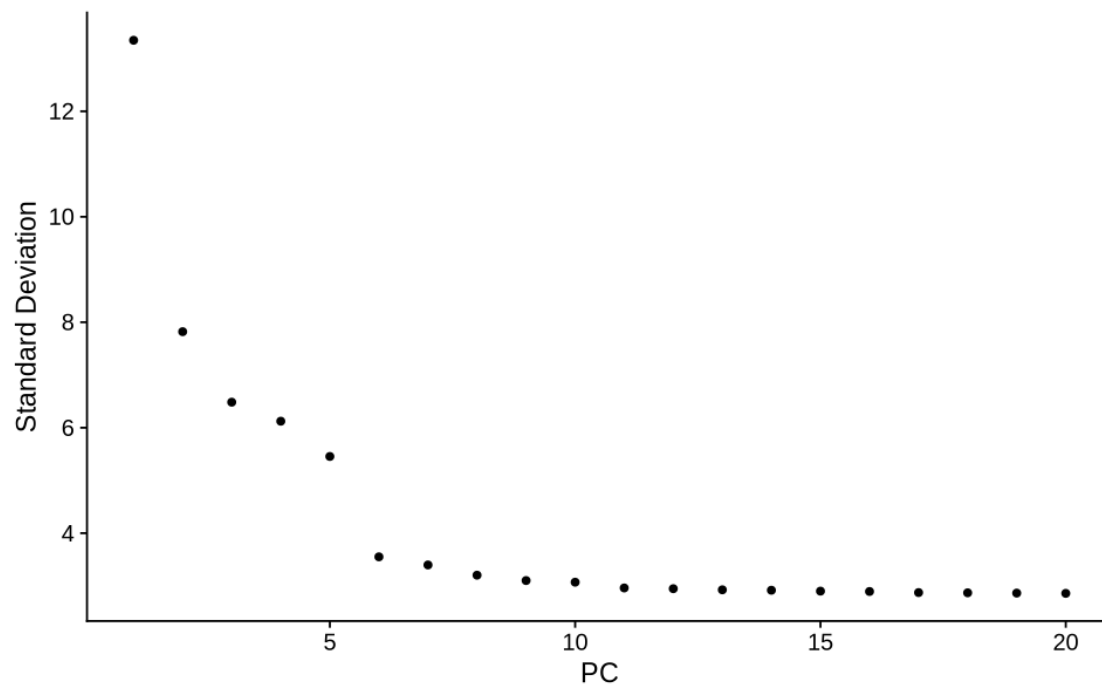
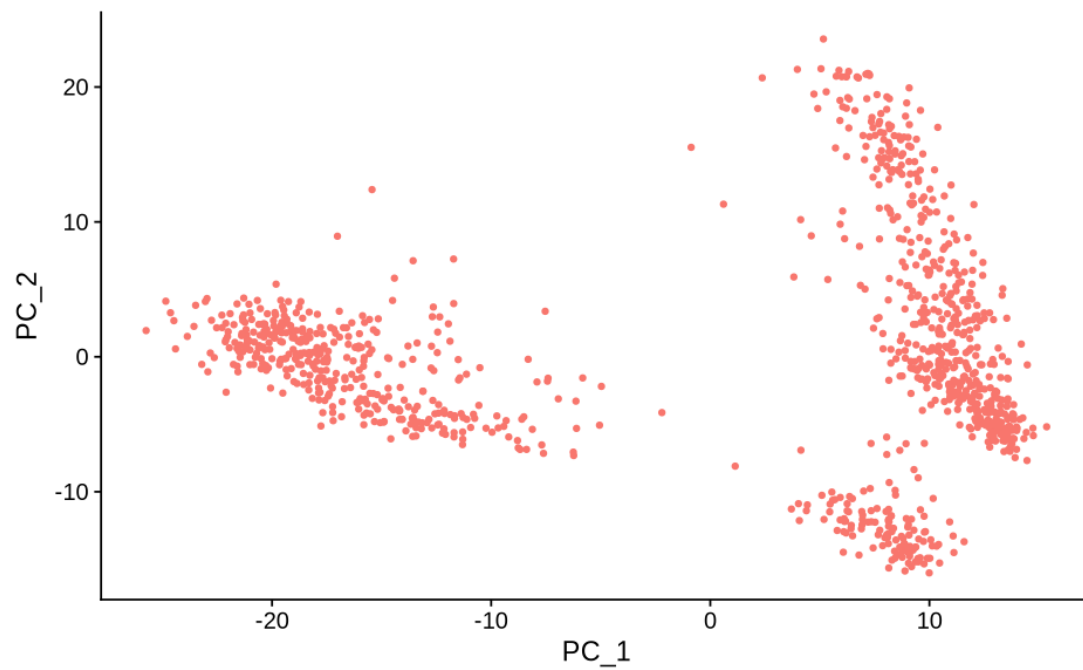
PC_ 5

Positive: S100A12, VCAN, S100A8, CD14, AC020656.1, S100A9, CD36, MGST1, MS4A6A,
CLEC4E

LGALS2, QPCT, RNASE6, CXCL8, CSF3R, ITGAM, LYZ, GPX1, KLRF1, FCN1
FGFBP2, GZMB, PLA2G7, GNLY, TYROBP, SPON2, KLRD1, CSTA, KCTD12, MNDA

Negative: NAP1L1, HES4, TCF7L2, CDKN1C, MS4A7, MS4A4A, FAM110A, SIGLEC10, CTSL,
CSF1R

BATF3, ADA, LRRC25, SMIM25, CD3D, TRAC, PABPC1, ZNF703, TUBA1B, HCK
BCL2A1, RPS2, RNASET2, FCGR3A, HMOX1, IL7R, UNC119, CD3G, INSIG1,
JUNB



```
[14]: objGenes_SoloTE <- FindNeighbors(objGenes_SoloTE, dims = 1:10, k.param = 20)
      objGenes_SoloTE <- FindClusters(objGenes_SoloTE, resolution = 1.6)
```

```
objGenes_SoloTE <- RunUMAP(objGenes_SoloTE, dims = 1:10)
DimPlot(objGenes_SoloTE, reduction = "umap")
```

Computing nearest neighbor graph

Computing SNN

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 33567

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7595

Number of communities: 11

Elapsed time: 0 seconds

Warning message:

"The default method for RunUMAP has changed from calling Python UMAP via
reticulate to the R-native UWOT using the cosine metric

To use Python UMAP via reticulate, set umap.method to 'umap-learn' and metric to
'correlation'

This message will be shown once per session"

14:20:58 UMAP embedding parameters a = 0.9922 b = 1.112

14:20:58 Read 1056 rows and found 10 numeric columns

14:20:58 Using Annoy for neighbor search, n_neighbors = 30

14:20:58 Building Annoy index with metric = cosine, n_trees = 50

0% 10 20 30 40 50 60 70 80 90 100%

[----|----|----|----|----|----|----|----|----|----|

*
*
*
*
*
*
*
*
*
*
*
*
*
*
*

```
14:20:58 Writing NN index file to temp file
/mnt/TEdataStorage_2T/tmp/RtmpsweA5r/filea57242f56b2d1

14:20:58 Searching Annoy index using 1 thread, search_k = 3000

14:20:58 Annoy recall = 100%

14:21:03 Commencing smooth kNN distance calibration using 1 thread
```

```

with target n_neighbors = 30

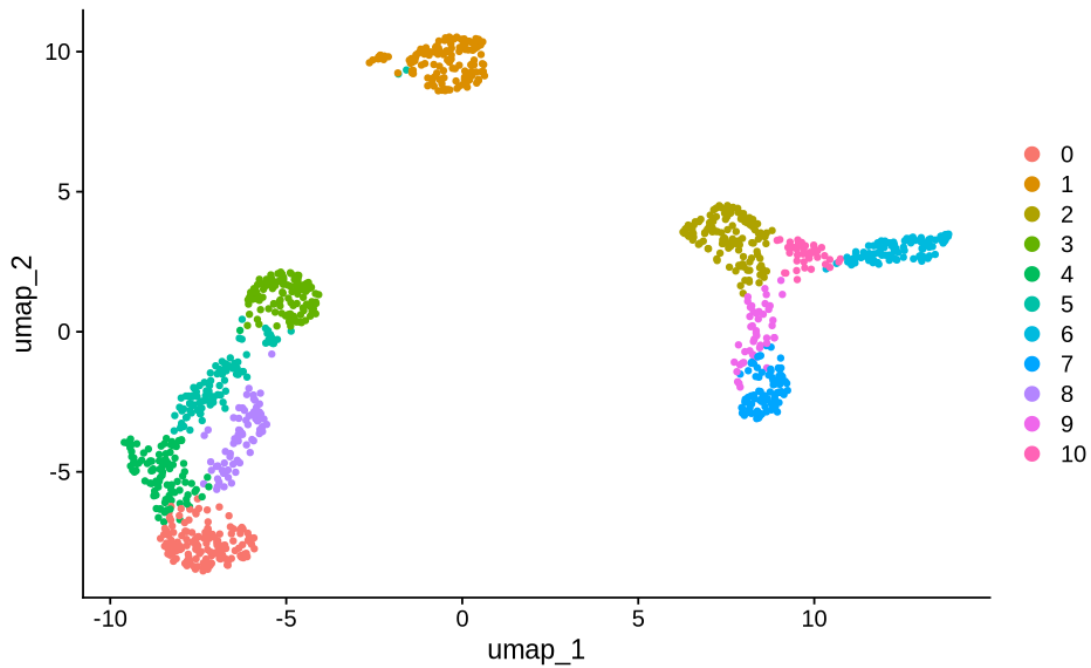
14:21:12 Initializing from normalized Laplacian + noise (using RSpectra)

14:21:12 Commencing optimization for 500 epochs, with 40676 positive edges

14:21:12 Using rng type: pcg

14:21:18 Optimization finished

```



```

[15]: objGenes_SoloTE$celltype <- celltypes$V2[match(Cells(objGenes_SoloTE),
↪celltypes$V1)]
table(objGenes_SoloTE$celltype)

```

B	Dendritic	Monocytes_CD14pos	Monocytes_FCGR3Apos
132	132	132	132
NK	T_CD4_Memory	T_CD4_Naive	T_CD8
132	132	132	132

```

[19]: options(repr.plot.width=10, repr.plot.height=7)

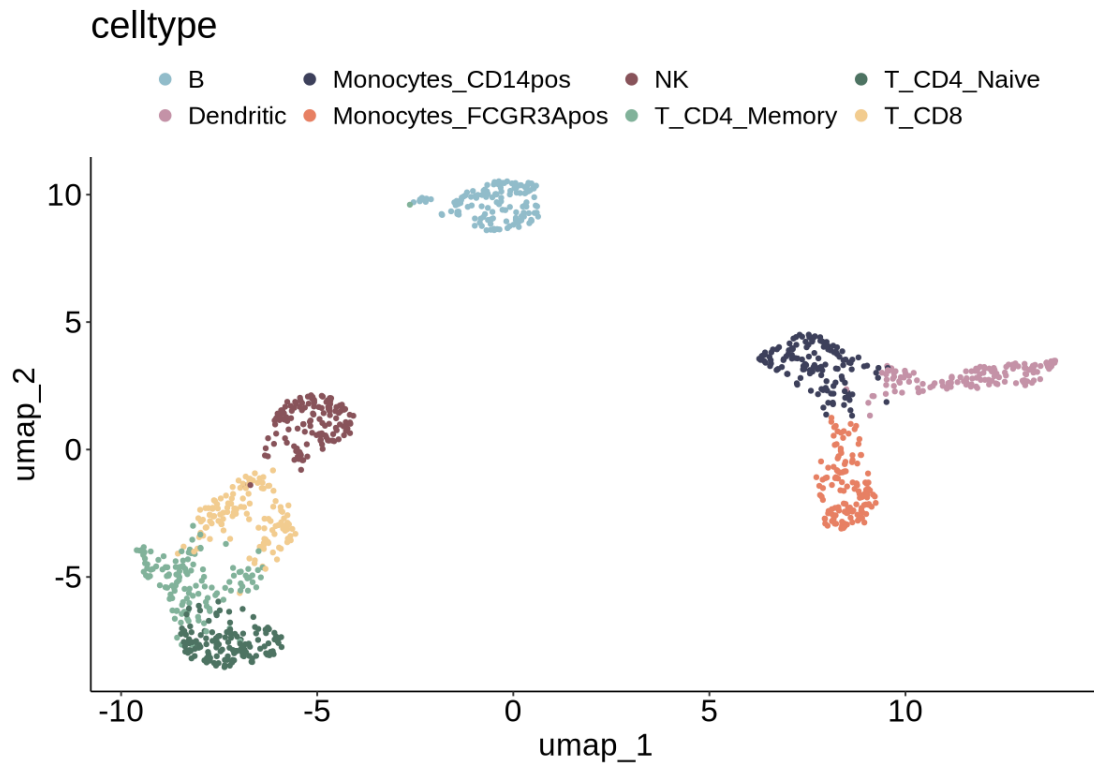
DimPlot(objGenes_SoloTE, reduction = "umap", group.by = "celltype",
cols=PBMCcelltypeColors,

```

```

    shuffle=T, pt.size = 1) +
  theme_pubr() +
  theme(text=element_text(size=20))
ggsave(paste0("figures_", dataset_id, "/umap_genes_celltypes_SoloTE_noplatelet.
  pdf"), device = "pdf", width=10, height=7)

```



```

[20]: saveRDS(objGenes_SoloTE, file=paste0("data_", dataset_id, "/"
  objGenes_SoloTE_noPlatelet.RDS"))

```

2 TEs

```

[21]: ### Normalize

objTE_SoloTE <- NormalizeData(objTE_SoloTE, normalization.method =
  "LogNormalize", scale.factor = 10000)

objTE_SoloTE <- FindVariableFeatures(objTE_SoloTE, selection.method = "vst",
  nfeatures = 4000)

# Identify the 10 most highly variable genes
top10 <- head(VariableFeatures(objTE_SoloTE), 10)

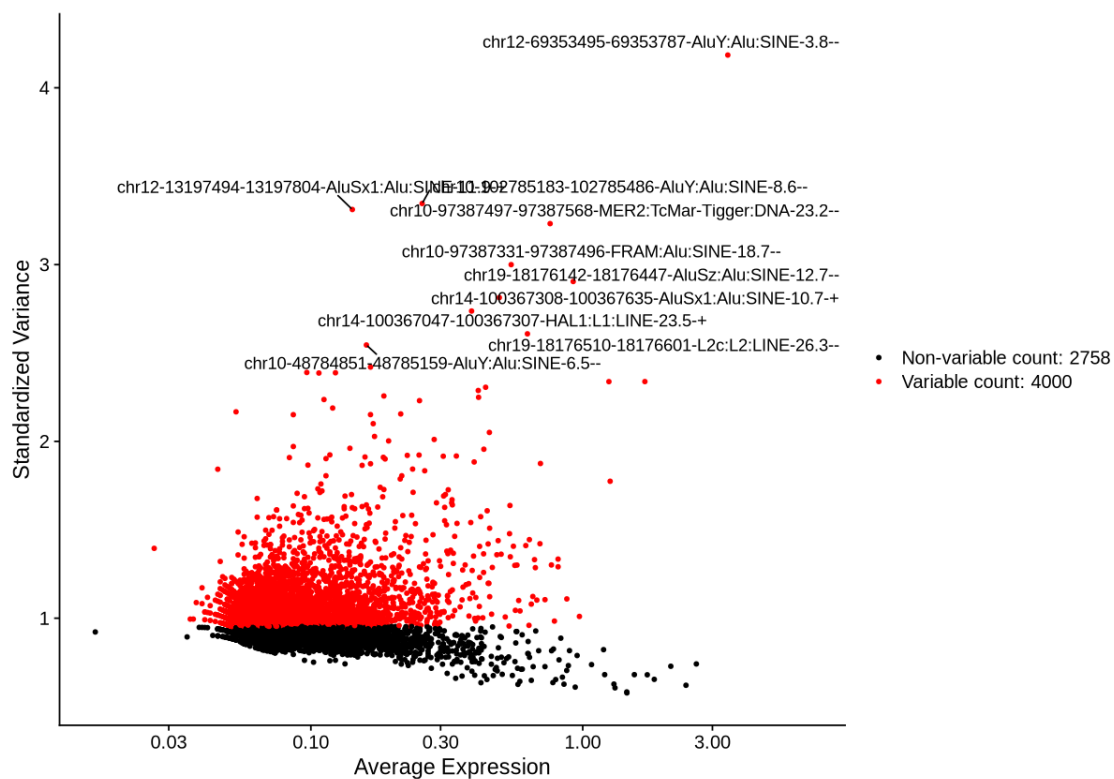
```

```
# plot variable features with and without labels
plot1 <- VariableFeaturePlot(objTE_SoloTE)
plot2 <- LabelPoints(plot = plot1, points = top10, repel = TRUE)
plot2
```

Normalizing layer: counts

Finding variable features for layer counts

When using repel, set xnudge and ynudge to 0 for optimal results



```
[22]: gc()
all.genes <- rownames(objTE_SoloTE)
objTE_SoloTE <- ScaleData(objTE_SoloTE) # on hvg
grep("MERVL", all.genes, value = T)[1:50]
```

		used	(Mb)	gc trigger	(Mb)	max used	(Mb)
A matrix: 2×6 of type dbl	Ncells	27969199	1493.8	63935878	3414.6	42920484	2292.3
	Vcells	250960557	1914.7	423138372	3228.3	422858482	3226.2

Centering and scaling data matrix

1. NA 2. NA 3. NA 4. NA 5. NA 6. NA 7. NA 8. NA 9. NA 10. NA 11. NA 12. NA 13. NA 14. NA
15. NA 16. NA 17. NA 18. NA 19. NA 20. NA 21. NA 22. NA 23. NA 24. NA 25. NA 26. NA
27. NA 28. NA 29. NA 30. NA 31. NA 32. NA 33. NA 34. NA 35. NA 36. NA 37. NA 38. NA
39. NA 40. NA 41. NA 42. NA 43. NA 44. NA 45. NA 46. NA 47. NA 48. NA 49. NA 50. NA

```
[23]: objTE_SoloTE <- RunPCA(objTE_SoloTE, features = VariableFeatures(object =  
  ↪objTE_SoloTE))  
  
DimPlot(objTE_SoloTE, reduction = "pca") + NoLegend()  
  
ElbowPlot(objTE_SoloTE)
```

PC_ 1

Positive: chr12-10378872-10379096-AluSz:Alu:SINE-12.9-+,
chr12-10379097-10379234-AluSx3:Alu:SINE-11.6-+,
chr17-35501071-35507907-Harlequin-int:ERV1:LTR-6.2-+,
chr12-10372259-10372584-MLT-int:ERVL-MaLR:LTR-19.9--,
chr11-118353525-118353830-AluSz:Alu:SINE-10.0--,
chr5-157233964-157234281-AluSx:Alu:SINE-13.4--,
chr10-72082577-72082882-AluJb:Alu:SINE-13.1-+,
chr10-72082410-72082576-MLT1A:ERVL-MaLR:LTR-22.8-+,
chr14-61505446-61505674-AluSp:Alu:SINE-10.9--,
chr14-91864778-91865070-AluSz:Alu:SINE-11.3--
chr14-61505675-61505828-MLT1A0:ERVL-MaLR:LTR-19.6--,
chr19-49485895-49486103-MIRb:MIR:SINE-31.9--,
chr4-40201946-40202244-AluJb:Alu:SINE-14.3--,
chr9-127964821-127965136-AluSx1:Alu:SINE-6.7-+,
chr4-142196866-142197150-AluSq2:Alu:SINE-7.7-+,
chr4-142192140-142192350-L1PA15:L1:LINE-13.0-+,
chr14-99234510-99234621-MIRb:MIR:SINE-33.0-+,
chr14-99179192-99179499-AluSz:Alu:SINE-8.1-+,
chr14-61339706-61339983-AluJo:Alu:SINE-15.5--,
chr14-22466278-22466376-L2c:L2:LINE-23.6--
chr12-9688765-9691861-L1M4:L1:LINE-23.7-+,
chr6-42003244-42003555-AluSp:Alu:SINE-11.9-+,
chr1-167506375-167506587-AluSx3:Alu:SINE-17.4--,
chr12-9083439-9086260-L1MA7:L1:LINE-21.1-+,
chr11-60985175-60985484-AluSq2:Alu:SINE-9.2--,
chr6-42045271-42045503-MIRb:MIR:SINE-26.9-+,
chr7-130932308-130932608-AluSz:Alu:SINE-11.3-+,
chr14-61329235-61329534-AluSx:Alu:SINE-9.3--,
chr12-7935007-7935138-AluJb:Alu:SINE-17.9--,
chr11-96311959-96312247-AluSx1:Alu:SINE-11.4-+
Negative: chr17-7581641-7581952-AluSz:Alu:SINE-7.4-+,

chr19-18176142-18176447-AluSz:Alu:SINE-12.7--,
 chr19-18176510-18176601-L2c:L2:LINE-26.3--,
 chr12-8138452-8138539-L2b:L2:LINE-28.8-- , chr7-106269678-106269812-MER5C1:hAT-
 Charlie:DNA-23.0-- , chr7-106269455-106269610-L4-B-Mam:RTE-X:LINE-29.8-+ ,
 chr12-69353495-69353787-AluY:Alu:SINE-3.8-- ,
 chr7-106259722-106261114-Tigger1:TcMar-Tigger:DNA-12.9-- ,
 chr7-106267018-106267090-L2a:L2:LINE-31.6-+ ,
 chr14-100367308-100367635-AluSx1:Alu:SINE-10.7-+
 chr9-27572377-27572618-MamRTE1:RTE-BovB:LINE-36.6-- ,
 chr14-100367047-100367307-HAL1:L1:LINE-23.5-+ ,
 chr1-36476656-36476968-AluJb:Alu:SINE-15.3-- ,
 chr7-106267561-106267870-AluYa5:Alu:SINE-1.3-+ ,
 chr6-37028614-37028907-AluJb:Alu:SINE-15.0-- ,
 chr9-134911343-134911650-AluSc8:Alu:SINE-10.4-- ,
 chr11-47381692-47381998-AluSq:Alu:SINE-10.1-- ,
 chr19-43666873-43667182-AluSz:Alu:SINE-13.6-- ,
 chr1-36476969-36477256-L2c:L2:LINE-31.7-+ ,
 chr11-60379229-60379459-L1ME3Cz:L1:LINE-26.9-+
 chr2-237719941-237720246-AluSz6:Alu:SINE-12.5-- ,
 chr12-10117248-10117551-AluSp:Alu:SINE-8.6-+ ,
 chr10-97387659-97387968-AluSp:Alu:SINE-7.4-+ ,
 chr22-17757133-17757426-AluJb:Alu:SINE-11.9-+ ,
 chr5-150409245-150409543-AluSx1:Alu:SINE-9.0-+ ,
 chr5-150409544-150409722-AluSx1:Alu:SINE-11.7-+ ,
 chr11-47382001-47382299-AluJb:Alu:SINE-18.1-- ,
 chr2-237720257-237720547-AluSz:Alu:SINE-13.1-- ,
 chr22-17756874-17757132-LTR37-int:ERV1:LTR-25.9-- ,
 chr16-12758881-12759192-AluJo:Alu:SINE-17.0-+
 PC_ 2
 Positive: chr5-142784924-142785203-AluY:Alu:SINE-6.1-- ,
 chr7-106259722-106261114-Tigger1:TcMar-Tigger:DNA-12.9-- ,
 chr5-142778431-142778754-L1ME3G:L1:LINE-27.4-- ,
 chr12-69353495-69353787-AluY:Alu:SINE-3.8-- ,
 chr7-106269455-106269610-L4-B-Mam:RTE-X:LINE-29.8-+ ,
 chr19-43654994-43655299-AluSx1:Alu:SINE-10.1-+ ,
 chr17-7581641-7581952-AluSz:Alu:SINE-7.4-+ ,
 chr2-142883498-142883932-L2a:L2:LINE-32.1-+ ,
 chr5-142787993-142788300-AluSz:Alu:SINE-9.4-- ,
 chr2-142883187-142883497-AluYe5:Alu:SINE-4.9--
 chr2-207528649-207528957-AluSq2:Alu:SINE-7.8-+ ,
 chr5-83529270-83529618-MER2:TcMar-Tigger:DNA-19.6-+ ,
 chr7-106269678-106269812-MER5C1:hAT-Charlie:DNA-23.0-- ,
 chr20-8417074-8417290-AluY:Alu:SINE-8.1-- ,
 chr19-43654732-43654906-L2b:L2:LINE-29.1-- ,
 chr19-43654954-43654993-MIR:MIR:SINE-29.8-- ,
 chr12-10117248-10117551-AluSp:Alu:SINE-8.6-+ ,
 chr22-42391663-42391969-AluSz:Alu:SINE-7.5-+ ,
 chr22-42391279-42391662-L2a:L2:LINE-26.1-- ,

chr2-218368322-218368627-AluSg:Alu:SINE-7.2-+
chr11-9242808-9243116-AluSg:Alu:SINE-8.7-+,
chr2-218368170-218368321-MIR:MIR:SINE-25.8--,
chr14-22466278-22466376-L2c:L2:LINE-23.6--,
chr1-36476656-36476968-AluJb:Alu:SINE-15.3--,
chr17-43233260-43233516-HERV17-int:ERV1:LTR-13.6-+,
chr17-2860595-2860905-AluSz:Alu:SINE-12.0--,
chr11-9242741-9242807-MamSINE1:tRNA-RTE:SINE-28.4--,
chr20-4024584-4025416-HAL1:L1:LINE-22.6-+,
chr10-70601547-70601845-AluSx1:Alu:SINE-11.0-+,
chr19-43659153-43659451-AluJo:Alu:SINE-15.1--
Negative: chr22-41924087-41924204-AluJr:Alu:SINE-18.5-+,
chr1-178856427-178856556-AluJr:Alu:SINE-15.6--,
chr22-41924672-41924979-AluSc5:Alu:SINE-5.2-+,
chr1-178856558-178856629-L1M5:L1:LINE-19.4--,
chr22-41924205-41924509-AluY:Alu:SINE-6.2-+,
chr15-57300905-57301111-L1MC5a:L1:LINE-27.0-+,
chr22-41924510-41924671-AluJr:Alu:SINE-18.5-+,
chr15-57301114-57301428-AluSx1:Alu:SINE-14.9-+,
chr19-17533128-17533436-AluSx1:Alu:SINE-12.0-+,
chr19-17532992-17533127-AluSz:Alu:SINE-14.7-+
chr2-100035752-100035902-MIRc:MIR:SINE-31.3-+,
chr6-90203129-90203416-AluJb:Alu:SINE-11.8-+,
chr6-32638390-32638696-AluJr:Alu:SINE-16.9-+,
chr1-178856801-178856873-Tigger9a:TcMar-Tigger:DNA-24.7--,
chr10-48784531-48784837-AluSc:Alu:SINE-12.6--,
chr8-24336302-24336609-AluY:Alu:SINE-6.5-+,
chr12-110662734-110663320-L1M5:L1:LINE-28.6-+,
chr12-110663321-110663616-AluJb:Alu:SINE-15.2-+,
chr10-48784851-48785159-AluY:Alu:SINE-6.5--,
chr2-100057043-100057344-AluSx1:Alu:SINE-9.6-+
chr4-85591032-85591344-AluSq2:Alu:SINE-14.8--,
chr2-60542747-60543053-AluSq:Alu:SINE-9.4-+,
chr10-48784838-48784850-L1MD2:L1:LINE-17.8--,
chr5-150405760-150406070-AluSx1:Alu:SINE-9.7--,
chr2-99871926-99872232-AluY:Alu:SINE-6.5-+,
chr2-99871709-99871922-L1PA17:L1:LINE-19.8-+,
chr3-112476673-112478768-L1MB5:L1:LINE-19.6--,
chr12-108636777-108637096-AluSz6:Alu:SINE-12.1--,
chr2-99966409-99966719-AluYa5:Alu:SINE-0.3-+,
chr5-150411957-150412264-AluSx:Alu:SINE-14.3--
PC_ 3
Positive: chr15-57300905-57301111-L1MC5a:L1:LINE-27.0-+,
chr22-41924672-41924979-AluSc5:Alu:SINE-5.2-+,
chr22-41924510-41924671-AluJr:Alu:SINE-18.5-+,
chr15-57301114-57301428-AluSx1:Alu:SINE-14.9-+,
chr22-41924087-41924204-AluJr:Alu:SINE-18.5-+,
chr22-41924205-41924509-AluY:Alu:SINE-6.2-+

chr19-17533128-17533436-AluSx1:Alu:SINE-12.0-+,
 chr1-178856427-178856556-AluJr:Alu:SINE-15.6--,
 chr1-178856558-178856629-L1M5:L1:LINE-19.4--,
 chr19-17532992-17533127-AluSz:Alu:SINE-14.7-+
 chr1-157679635-157679858-AluSx:Alu:SINE-12.9-+,
 chr12-110662734-110663320-L1M5:L1:LINE-28.6-+,
 chr12-110663321-110663616-AluJb:Alu:SINE-15.2-+,
 chr2-60542747-60543053-AluSq:Alu:SINE-9.4-+,
 chr1-178856801-178856873-Tigger9a:TcMar-Tigger:DNA-24.7--,
 chr19-17532602-17532829-MIRb:MIR:SINE-32.1-+,
 chr4-85591032-85591344-AluSq2:Alu:SINE-14.8--,
 chr3-112476673-112478768-L1MB5:L1:LINE-19.6--,
 chr12-108637097-108637289-MIR:MIR:SINE-27.1--,
 chr12-108636777-108637096-AluSz6:Alu:SINE-12.1--
 chr2-100035752-100035902-MIRc:MIR:SINE-31.3-+,
 chr19-2645201-2645538-AluSx1:Alu:SINE-14.4--,
 chr2-100057043-100057344-AluSx1:Alu:SINE-9.6-+,
 chr9-123233919-123234228-AluY:Alu:SINE-7.1-+,
 chr6-90203129-90203416-AluJb:Alu:SINE-11.8-+,
 chr4-85608552-85608859-AluSx:Alu:SINE-12.4--,
 chr2-45712138-45712455-AluSq:Alu:SINE-11.0--,
 chr8-80164602-80164927-AluSz6:Alu:SINE-11.0-+,
 chr2-45751810-45752115-AluYa5:Alu:SINE-1.1--,
 chr11-9714971-9715278-AluJb:Alu:SINE-15.3--
 Negative: chr8-100700358-100700666-AluSq:Alu:SINE-8.4-+,
 chr8-100700309-100700357-Tigger17a:TcMar-Tigger:DNA-19.8--,
 chr6-158650661-158650969-AluSx1:Alu:SINE-11.7-+,
 chr7-131045951-131046252-AluSx1:Alu:SINE-8.3-+,
 chr7-131045767-131045950-LTR45B:ERV1:LTR-16.4-+,
 chr15-92908837-92909140-AluSx1:Alu:SINE-13.8-+,
 chr6-158804767-158805065-L1PA2:L1:LINE-0.7--,
 chr7-131044479-131044766-AluY:Alu:SINE-9.4--,
 chr7-131078283-131078578-AluSg:Alu:SINE-8.4--,
 chr6-158805103-158805362-AluSc5:Alu:SINE-8.1-+
 chr7-139104432-139104746-AluSx1:Alu:SINE-6.7-+,
 chr14-61505675-61505828-MLT1A0:ERVL-MaLR:LTR-19.6--,
 chr14-61505446-61505674-AluSp:Alu:SINE-10.9--,
 chr6-106521712-106522031-AluSx4:Alu:SINE-10.2--,
 chr17-78804180-78804514-AluSx:Alu:SINE-9.4-+,
 chr7-131071402-131071494-MIRb:MIR:SINE-26.9-+,
 chr14-99179192-99179499-AluSz:Alu:SINE-8.1-+,
 chr11-118353525-118353830-AluSz:Alu:SINE-10.0--,
 chr6-111449095-111450199-L1ME1:L1:LINE-24.4-+,
 chr8-100703768-100704075-AluSx1:Alu:SINE-10.4-+
 chr19-53557258-53557547-MLT1A0:ERVL-MaLR:LTR-23.8--,
 chr1-24922320-24922797-Charlie4a:hAT-Charlie:DNA-19.0-+,
 chr19-53550561-53550836-AluSx1:Alu:SINE-11.6--,
 chr1-156121076-156121320-AluJb:Alu:SINE-16.3--

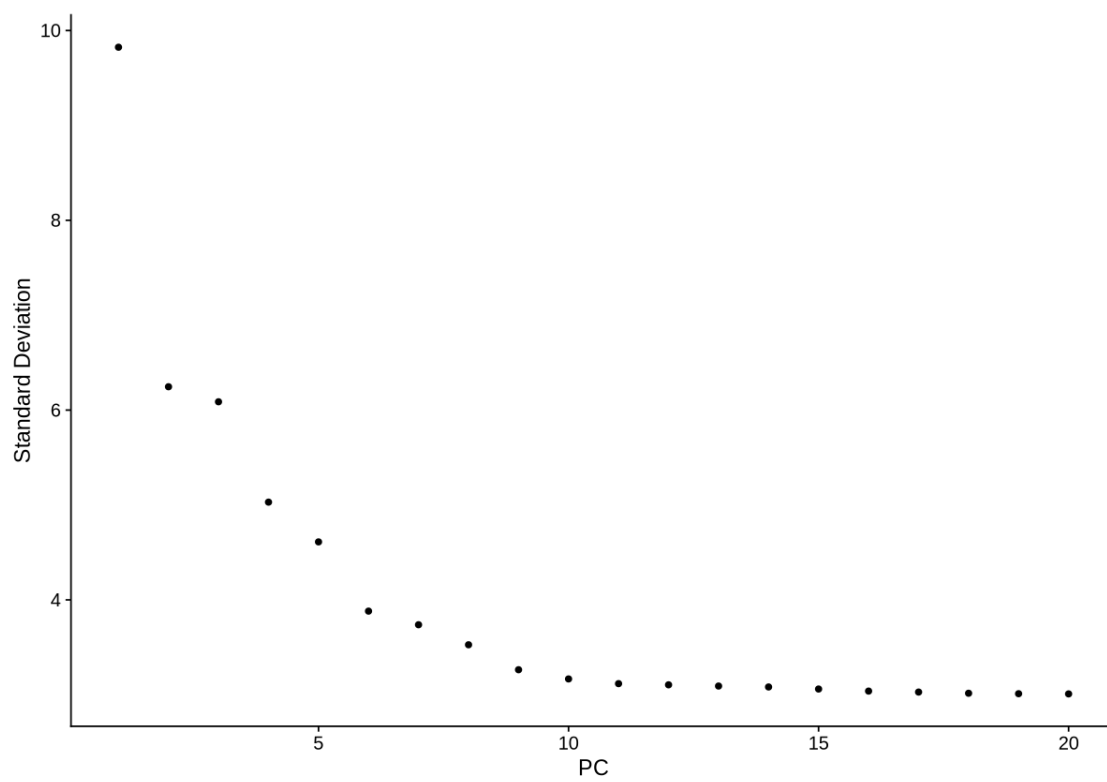
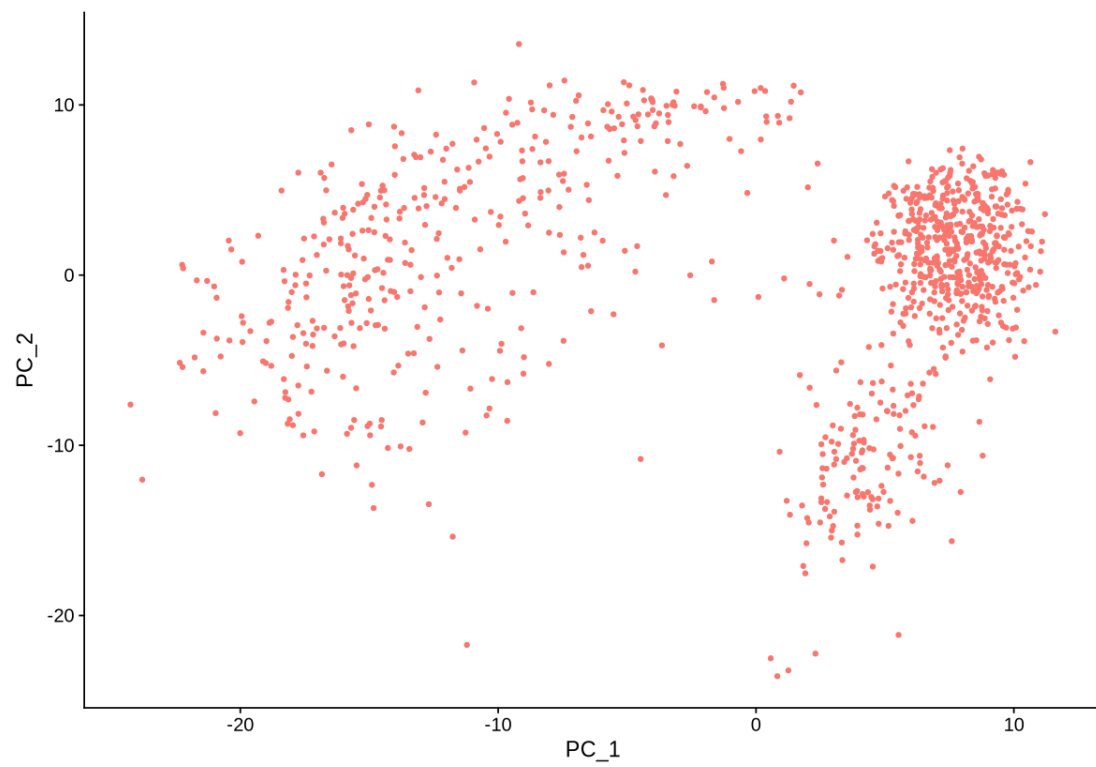
chr4-142196866-142197150-AluSq2:Alu:SINE-7.7-+,
 chr1-156117995-156118122-AluJr:Alu:SINE-19.5--,
 chr19-53557123-53557257-AluJo:Alu:SINE-17.3--,
 chr19-53557000-53557122-AluJb:Alu:SINE-15.7--,
 chr7-130932308-130932608-AluSz:Alu:SINE-11.3-+,
 chr7-131044822-131045286-L1ME1:L1:LINE-21.1--
 PC_ 4
 Positive: chr17-58658800-58658987-AluJr:Alu:SINE-22.9--,
 chr12-69353495-69353787-AluY:Alu:SINE-3.8--,
 chr5-55950255-55950567-AluSx1:Alu:SINE-8.0-+,
 chr19-53550561-53550836-AluSx1:Alu:SINE-11.6--,
 chr1-44730545-44730685-AluJr:Alu:SINE-16.3-+,
 chr5-80140048-80140332-AluJb:Alu:SINE-17.5-+,
 chr15-65296541-65296879-AluY:Alu:SINE-4.4-+,
 chr1-44819664-44819825-AluSx:Alu:SINE-24.1--,
 chr19-53550844-53551130-AluSg:Alu:SINE-10.8--,
 chr17-43233260-43233516-HERV17-int:ERV1:LTR-13.6-+
 chr4-108086263-108086388-FLAM-A:Alu:SINE-14.3--,
 chr11-61162162-61162256-MER57-int:ERV1:LTR-20.0-+,
 chr4-108086399-108086457-L2b:L2:LINE-28.8--,
 chr1-11909187-11909506-AluSx:Alu:SINE-5.6-+,
 chr14-75257894-75258210-AluSx1:Alu:SINE-9.5--,
 chr11-61162263-61162422-MER57B2:ERV1:LTR-11.9-+,
 chr1-36476656-36476968-AluJb:Alu:SINE-15.3--,
 chr1-44819555-44819664-AluSx3:Alu:SINE-20.9--,
 chr4-108086517-108086685-MIR:MIR:SINE-26.6-+,
 chr17-58658388-58658680-AluSz:Alu:SINE-9.9--
 chr2-121531314-121531609-AluYe5:Alu:SINE-4.4-+,
 chr1-44721624-44721711-AluJb:Alu:SINE-20.4-+, chr1-11908990-11909143-MLT1J:ERVL-
 MaLR:LTR-19.6-- , chr5-83529270-83529618-MER2:TcMar-Tigger:DNA-19.6-+,
 chr1-44730709-44730899-AluSx4:Alu:SINE-16.8-+,
 chr12-7932178-7932472-AluSx1:Alu:SINE-12.5--,
 chr17-66404743-66405049-AluSz:Alu:SINE-8.6--,
 chr17-43272629-43272830-HERVIP10FH-int:ERV1:LTR-20.1-+,
 chr12-7923769-7924066-AluSp:Alu:SINE-11.1--,
 chr19-53557000-53557122-AluJb:Alu:SINE-15.7--
 Negative: chr14-22466278-22466376-L2c:L2:LINE-23.6--,
 chr17-2810523-2810828-AluSg:Alu:SINE-7.9--,
 chr17-2860595-2860905-AluSz:Alu:SINE-12.0--,
 chr17-2810829-2811227-L3:CR1:LINE-32.9--,
 chr10-70601547-70601845-AluSx1:Alu:SINE-11.0-+,
 chr17-2860906-2861412-L2a:L2:LINE-29.6-+,
 chr2-144493871-144494176-AluY:Alu:SINE-3.3-+,
 chr12-9833927-9834338-LTR7:ERV1:LTR-7.4--,
 chr6-144824898-144825083-AluJb:Alu:SINE-16.8--,
 chr6-144824615-144824739-AluJb:Alu:SINE-11.1--
 chr8-124698502-124698821-AluSz6:Alu:SINE-9.4--,
 chr7-77604210-77604513-AluSx:Alu:SINE-10.5-- ,

chr15-94361353-94361488-MIRb:MIR:SINE-33.1-+,
chr12-10323850-10324161-AluSx1:Alu:SINE-14.4--,
chr7-77605946-77606253-AluSq2:Alu:SINE-14.5--,
chr16-31327306-31327595-AluJo:Alu:SINE-16.5-+, chr12-10372259-10372584-MLT-
int:ERVL-MaLR:LTR-19.9-- , chr1-91753105-91753418-AluJb:Alu:SINE-14.7-+,
chr15-94236633-94236724-L2a:L2:LINE-30.7-- ,
chr6-144825944-144826160-L1PA4:L1:LINE-4.2-+
chr11-36388998-36389311-L1ME4b:L1:LINE-28.4-- ,
chr14-92588215-92588524-AluSp:Alu:SINE-11.1-- ,
chr11-36388691-36388997-AluY:Alu:SINE-4.9-- ,
chr12-10324181-10324232-L1MD:L1:LINE-28.3-- ,
chr12-10378872-10379096-AluSz:Alu:SINE-12.9-+,
chr14-92588535-92588698-MLT1A0:ERVL-MaLR:LTR-16.1-- ,
chr12-10379097-10379234-AluSx3:Alu:SINE-11.6-+,
chr19-47311230-47311526-AluSc:Alu:SINE-11.8-+,
chr1-235801820-235801873-MamRTE1:RTE-BovB:LINE-26.4-+,
chr7-139898492-139898610-AluSx:Alu:SINE-5.0--

PC_ 5

Positive: chr19-47319300-47319605-AluSz6:Alu:SINE-14.1-- ,
chr19-47311230-47311526-AluSc:Alu:SINE-11.8-+,
chr19-47319606-47319654-MamRep38:hAT:DNA-27.7-+,
chr14-100367047-100367307-HAL1:L1:LINE-23.5-+,
chr14-100367308-100367635-AluSx1:Alu:SINE-10.7-+,
chr4-108086399-108086457-L2b:L2:LINE-28.8-- ,
chr19-47316898-47317221-AluJr4:Alu:SINE-16.7-+,
chr4-108086263-108086388-FLAM-A:Alu:SINE-14.3-- ,
chr4-108086517-108086685-MIR:MIR:SINE-26.6-+,
chr14-100350119-100350416-AluSx1:Alu:SINE-9.4-+
chr8-124698502-124698821-AluSz6:Alu:SINE-9.4-- ,
chr1-40345122-40345402-AluJr:Alu:SINE-13.2-+,
chr4-142192140-142192350-L1PA15:L1:LINE-13.0-+,
chr11-60379229-60379459-L1ME3Cz:L1:LINE-26.9-+,
chr4-142196866-142197150-AluSq2:Alu:SINE-7.7-+,
chr4-142333719-142334938-L1MA10:L1:LINE-17.9-- ,
chr17-7581641-7581952-AluSz:Alu:SINE-7.4-+,
chr6-144824615-144824739-AluJb:Alu:SINE-11.1-- ,
chr6-144824898-144825083-AluJb:Alu:SINE-16.8-- ,
chr2-197903803-197904102-AluSq2:Alu:SINE-8.7--
chr13-40623820-40624125-AluJr:Alu:SINE-13.7-+,
chr11-96198729-96198915-MLT1J:ERVL-MaLR:LTR-24.9-+,
chr11-96198916-96199222-AluSg7:Alu:SINE-8.8-+,
chr19-49485895-49486103-MIRb:MIR:SINE-31.9-- ,
chr11-60380158-60380307-L1ME3Cz:L1:LINE-27.3-+,
chr17-64340817-64341114-AluSp:Alu:SINE-7.4-+,
chr5-55950255-55950567-AluSx1:Alu:SINE-8.0-+,
chr7-139898492-139898610-AluSx:Alu:SINE-5.0-- ,
chr5-80140048-80140332-AluJb:Alu:SINE-17.5-+,
chr19-47316623-47316779-L1MB5:L1:LINE-20.0--

Negative: chr14-22466278-22466376-L2c:L2:LINE-23.6--,
 chr7-95412837-95412996-L1M5:L1:LINE-34.0--,
 chr7-95412997-95413300-AluJr:Alu:SINE-16.4-+,
 chr7-95412791-95412835-L2a:L2:LINE-11.1-+,
 chr10-70601547-70601845-AluSx1:Alu:SINE-11.0-+,
 chr7-95412678-95412759-HAL1:L1:LINE-26.8-+,
 chr16-31327306-31327595-AluJo:Alu:SINE-16.5-+,
 chr16-31327041-31327196-L2b:L2:LINE-31.0--,
 chr12-10323850-10324161-AluSx1:Alu:SINE-14.4-- , chr12-10372259-10372584-MLT-
 int:ERVL-MaLR:LTR-19.9--
 chr15-94236633-94236724-L2a:L2:LINE-30.7--,
 chr12-9833927-9834338-LTR7:ERV1:LTR-7.4--,
 chr7-77604210-77604513-AluSx:Alu:SINE-10.5--,
 chr1-91753105-91753418-AluJb:Alu:SINE-14.7-+,
 chr7-77605946-77606253-AluSq2:Alu:SINE-14.5--,
 chr15-94361353-94361488-MIRb:MIR:SINE-33.1-+,
 chr20-36645058-36645377-AluJr:Alu:SINE-13.8-+,
 chr12-10379097-10379234-AluSx3:Alu:SINE-11.6-+,
 chr12-10324181-10324232-L1MD:L1:LINE-28.3--,
 chr12-10378872-10379096-AluSz:Alu:SINE-12.9-+
 chr17-58321598-58322257-L2b:L2:LINE-29.2-+,
 chr17-58321298-58321597-AluSx1:Alu:SINE-10.0--,
 chr12-13197494-13197804-AluSx1:Alu:SINE-11.9-+,
 chr17-10113913-10114221-AluJb:Alu:SINE-13.9--,
 chr17-58658800-58658987-AluJr:Alu:SINE-22.9--,
 chr14-99285039-99285217-AluSx1:Alu:SINE-13.4--,
 chr22-38207647-38207827-AluJo:Alu:SINE-16.0--,
 chr6-158650661-158650969-AluSx1:Alu:SINE-11.7-+,
 chr15-94390718-94391029-AluSx1:Alu:SINE-10.9--,
 chr19-50336106-50336407-AluSx:Alu:SINE-11.6--



```
[18]: objTE_SoloTE <- FindNeighbors(objTE_SoloTE, dims = 1:12, k.param = 20)
objTE_SoloTE <- FindClusters(objTE_SoloTE, resolution = 1)
objTE_SoloTE <- RunUMAP(objTE_SoloTE, dims = 1:12)
DimPlot(objTE_SoloTE, reduction = "umap") +
  theme_void() +
  theme(text=element_text(size=20))
```

Computing nearest neighbor graph

Computing SNN

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 36390

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7881

Number of communities: 9

Elapsed time: 0 seconds

Warning message:

"The default method for RunUMAP has changed from calling Python UMAP via
reticulate to the R-native UWOT using the cosine metric

To use Python UMAP via reticulate, set umap.method to 'umap-learn' and metric to
'correlation'

This message will be shown once per session"

16:13:43 UMAP embedding parameters a = 0.9922 b = 1.112

16:13:43 Read 1056 rows and found 12 numeric columns

16:13:43 Using Annoy for neighbor search, n_neighbors = 30

16:13:43 Building Annoy index with metric = cosine, n_trees = 50

0% 10 20 30 40 50 60 70 80 90 100%

[----|----|----|----|----|----|----|----|----|----|

*
*
*
*
*


```
16:13:43 Writing NN index file to temp file
```

/mnt/TEdataStorage_2T/tmp/RtmptH78Gg/filead2022137b1cf

16:13:43 Searching Annoy index using 1 thread, search_k = 3000

16:13:44 Annoy recall = 100%

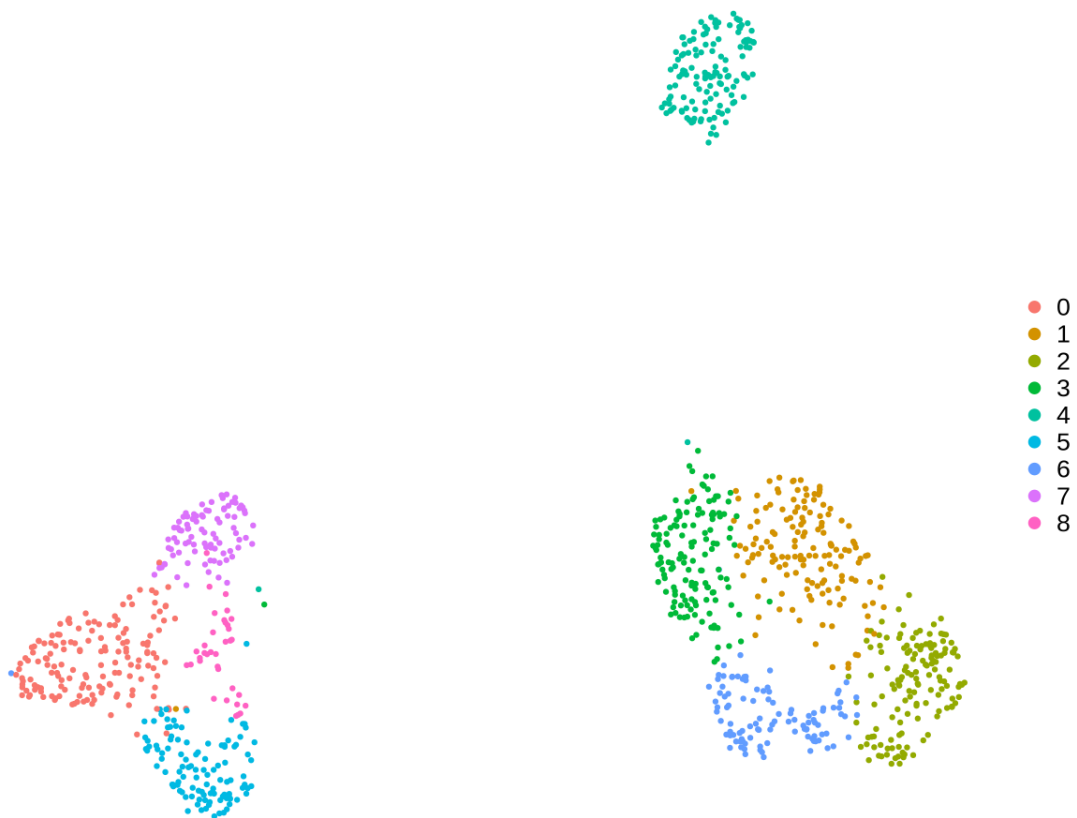
16:13:47 Commencing smooth kNN distance calibration using 1 thread
with target n_neighbors = 30

16:13:54 Initializing from normalized Laplacian + noise (using RSpectra)

16:13:54 Commencing optimization for 500 epochs, with 40348 positive edges

16:13:54 Using rng type: pcg

16:13:59 Optimization finished



3 Checkpoint

```
[19]: #saveRDS(objTE_SoloTE, paste0("data_", dataset_id, "/soloTE_", dataset_id, "_",  
    ↪ "_seuratObj_noPlatelet.RDS"))  
#objTE_SoloTE <- readRDS(paste0("data_", dataset_id, "/soloTE_", dataset_id, "_",  
    ↪ "_seuratObj.RDS"))
```

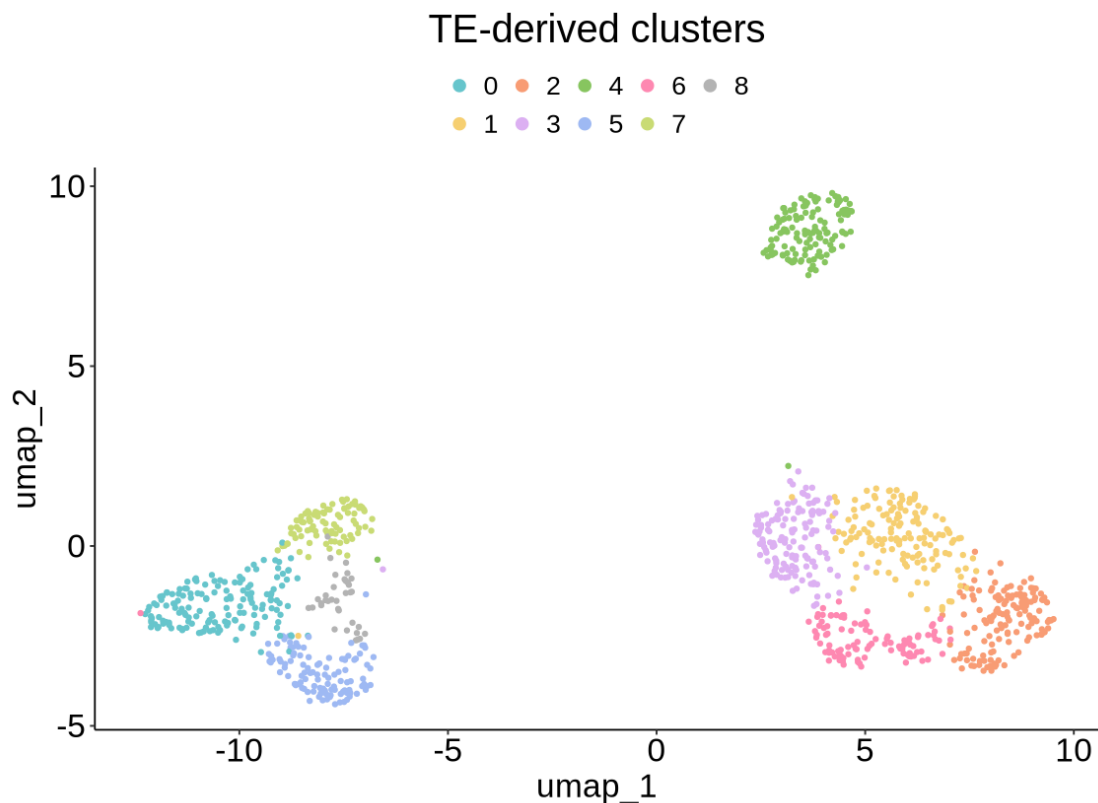
```
[ ]: objGenes_SoloTE <- readRDS(paste0("data_", sample_id, "/"  
    ↪ objGenes_SoloTE_noPlatelet.RDS"))
```

```
[20]: #save.image("workspaces/10xPBMC_SoloTE_260119_noplatelet.Rdata")
```

```
[2]: load("workspaces/10xPBMC_SoloTE_260119_noplatelet.Rdata")
```

4 Gene - TE comparison

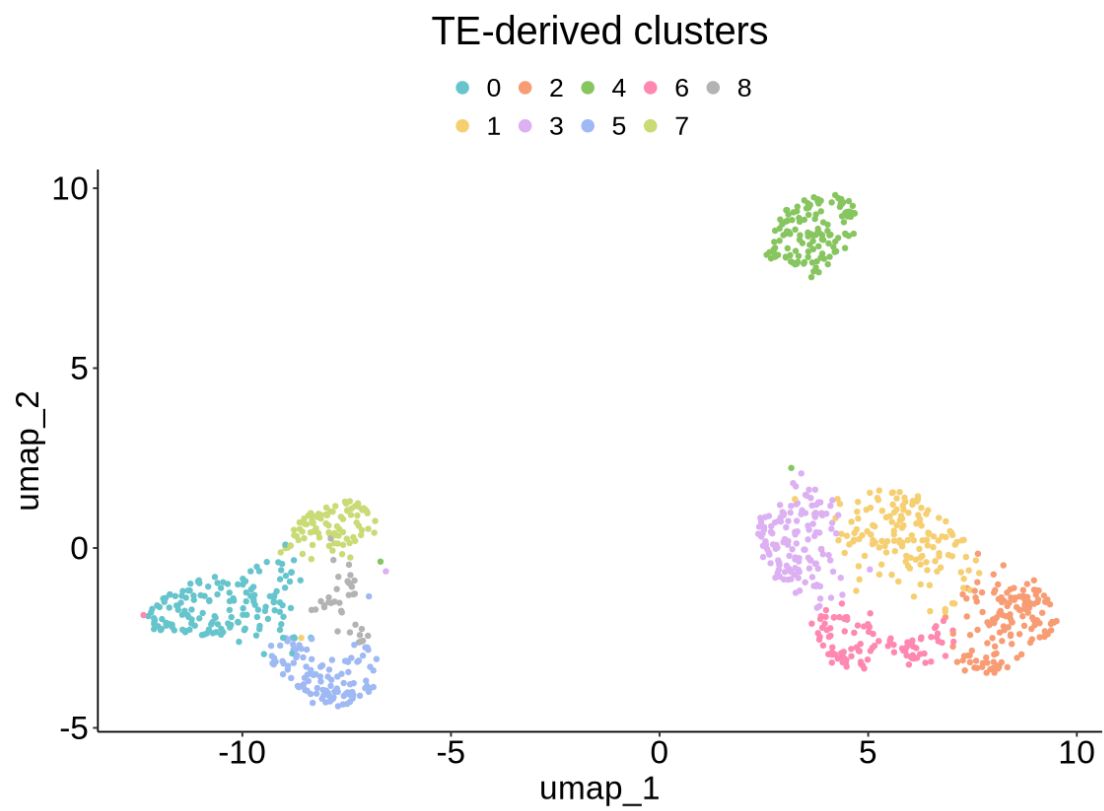
```
[3]: options(repr.plot.width=9.5, repr.plot.height=7)  
library(ggpubr)  
library(rcartocolor)  
  
DimPlot(objTE_SoloTE, reduction = "umap",  
    shuffle=T, pt.size = 1) +  
    ggtitle("TE-derived clusters") +  
    scale_color_carto_d(palette="Pastel") +  
    theme_pubr() +  
    theme(text=element_text(size=20), plot.title = element_text(hjust=0.5))  
ggsave(paste0("figures_", dataset_id, "/"  
    ↪ umap_TEs_locus_clusters_SoloTE_Pastel_noplatelet.pdf"), device = "pdf",  
    ↪ width=9.5, height=7)
```

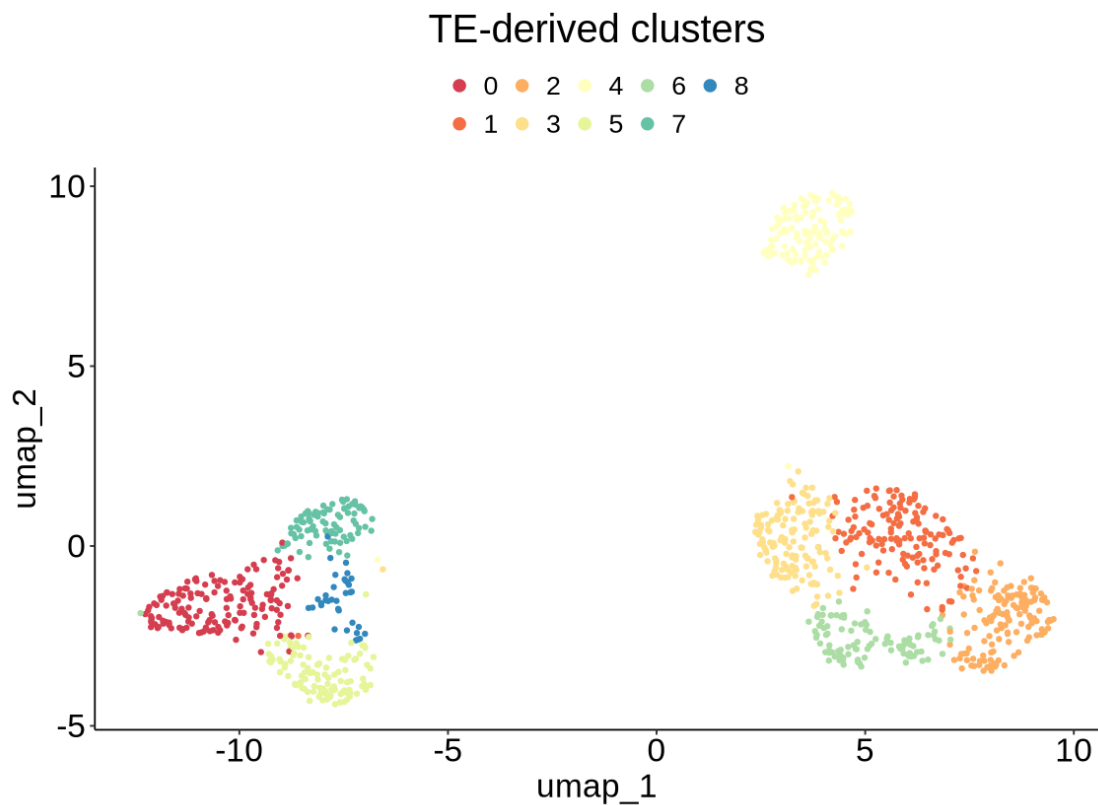


```
[4]: options(repr.plot.width=9.5, repr.plot.height=7)
library(ggpubr)
library(rcartocolor)

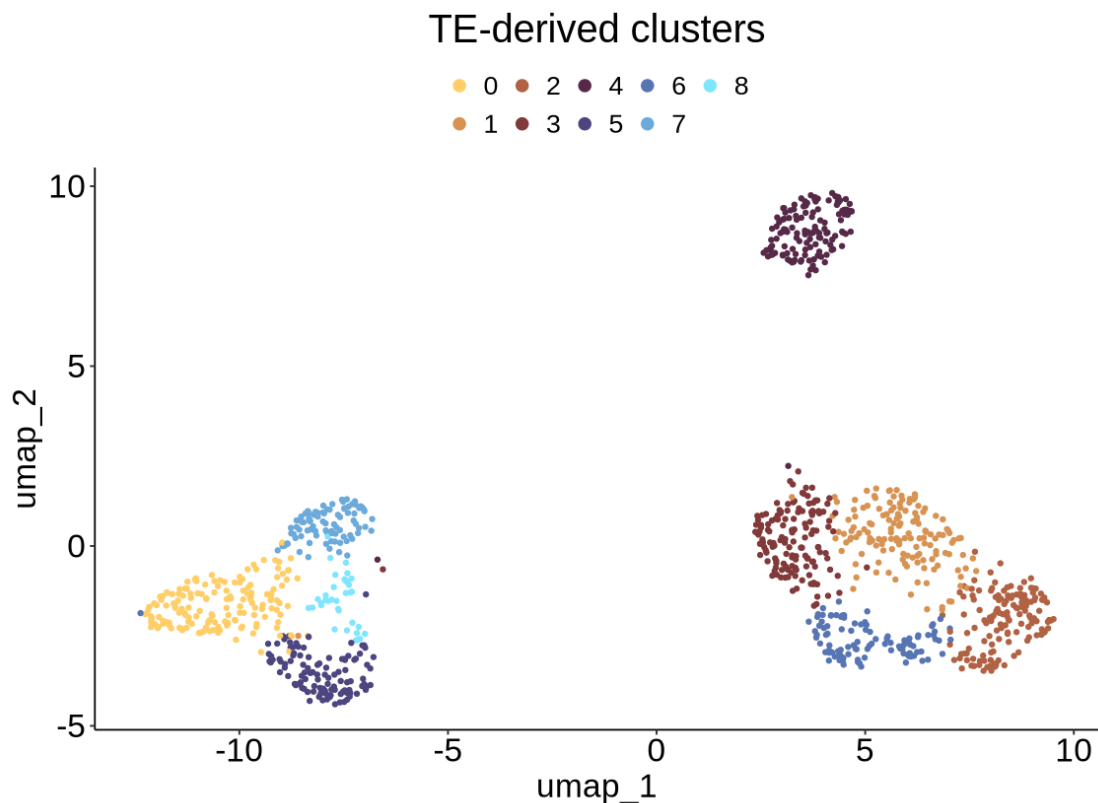
DimPlot(objTE_SoloTE, reduction = "umap",
        shuffle=T, pt.size = 1) +
  ggtitle("TE-derived clusters") +
  scale_color_carto_d(palette="Pastel") +
  theme_pubr() +
  theme(text=element_text(size=20), plot.title = element_text(hjust=0.5))
ggsave(paste0("figures_", dataset_id, "/umap_TEs_locus_clusters_SoloTE_noplatlet.
  pdf"), device = "pdf", width=9.5, height=7)

DimPlot(objTE_SoloTE, reduction = "umap",
        shuffle=T, pt.size = 1) +
  ggtitle("TE-derived clusters") +
  scale_color_brewer(palette="Spectral") +
  theme_pubr() +
  theme(text=element_text(size=20), plot.title = element_text(hjust=0.5))
```





```
[5]: options(repr.plot.width=9.5, repr.plot.height=7)
library(ggpubr)
library(scico)
DimPlot(objTE_SoloTE, reduction = "umap",
  shuffle=T, pt.size = 1) +
  ggtitle("TE-derived clusters") +
  scale_color_manual(values=
    ↪scico(length(unique(objTE_SoloTE$seurat_clusters)), palette = 'managua')) +
  theme_pubr() +
  theme(text=element_text(size=20), plot.title = element_text(hjust=0.5))
ggsave(paste0("figures_", dataset_id, "/"
  ↪umap_TEs_locus_clusters_SoloTE_scico_noplatelet.pdf"), device = "pdf",
  ↪width=9.5, height=7)
```



```
[6]: scico(30, palette = 'managua')
#> [1] "#190C64" "#1C176B" "#202272" "#212B79" "#243580" "#263D86" "#29478B"
#> [8] "#2C5091" "#2F5996" "#33619A" "#37699D" "#3D71A0" "#4479A1" "#4D81A2"
#> [15] "#5688A4" "#608EA2" "#6B94A1" "#77999F" "#839E9C" "#90A198" "#9BA495"
#> [22] "#A9A895" "#B7AD96" "#C7B59C" "#D7BEA6" "#E5C9B3" "#F0D4C3" "#F7DFD3"
#> [29] "#FCE9E3" "#FEF2F2"
```

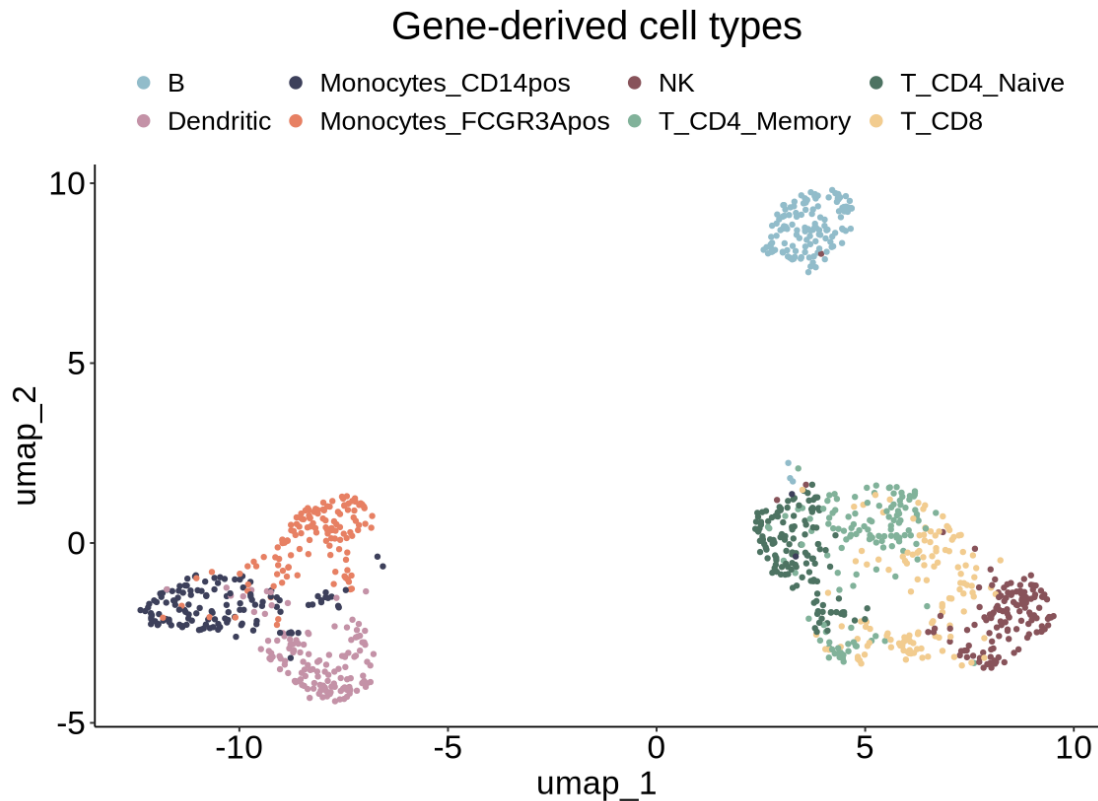
```
1. '#FFCE66' 2. '#F4BD61' 3. '#E9AC5B' 4. '#DE9C56' 5. '#D38D50' 6. '#C97F4C'
7. '#BE7148' 8. '#B36444' 9. '#A7583F' 10. '#9B4C3D' 11. '#8E423A' 12. '#7F3839'
13. '#712F39' 14. '#65293D' 15. '#5B2843' 16. '#54294D' 17. '#4F305C' 18. '#4B396B'
19. '#4C447D' 20. '#4C518E' 21. '#4F5D9C' 22. '#536BAA' 23. '#5878B6' 24. '#5D86C1'
25. '#6395CC' 26. '#68A4D5' 27. '#6EB4E0' 28. '#74C4EA' 29. '#7AD5F4' 30. '#80E6FF'
```

```
[7]: objTE_SoloTE$celltype <- celltypes$V2[match(Cells(objTE_SoloTE), celltypes$V1)]
table(objTE_SoloTE$celltype)
```

B	Dendritic	Monocytes_CD14pos	Monocytes_FCGR3Apos
132	132	132	132
NK	T_CD4_Memory	T_CD4_Naive	T_CD8
132	132	132	132

```
[8]: options(repr.plot.width=9.5, repr.plot.height=7)

DimPlot(objTE_SoloTE, reduction = "umap", group.by = "celltype",
        cols=PBMCcelltypeColors,
        shuffle=T, pt.size = 1) +
  ggtitle("Gene-derived cell types") +
  theme_pubr() +
  theme(text=element_text(size=20), plot.title = element_text(hjust=0.5))
ggsave(paste0("figures_",dataset_id,"/
↳umap_TEs_locus_celltypes_SoloTE_noplatelet.pdf"), device = "pdf", width=9.5,
↳height=7)
```



```
[9]: clustersResList <- list()
identical(Cells(objGenes_SoloTE), Cells(objTE_SoloTE)) # TRUE, same cells in
↳same order

for(res in seq(0.5, 2, by=0.1)){
  print(res)

  clusters_genes <- FindClusters(objGenes_SoloTE, resolution = res)
  clusters_TEs <- FindClusters(objTE_SoloTE, resolution = res)
```



```
clustersResList[[as.character(res)]] <-  
  cbind(clusters_genes$seurat_clusters, clusters_TEs$seurat_clusters)  
  #  
}
```

TRUE

[1] 0.5

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 33567

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.8919

Number of communities: 8

Elapsed time: 0 seconds

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 36390

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.8532

Number of communities: 8

Elapsed time: 0 seconds

[1] 0.6

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 33567

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.8792

Number of communities: 8

Elapsed time: 0 seconds

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 36390

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.8397

Number of communities: 8

Elapsed time: 0 seconds

[1] 0.7

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 33567

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.8666
Number of communities: 8
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 36390

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.8266
Number of communities: 8
Elapsed time: 0 seconds
[1] 0.8
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 33567

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.8540
Number of communities: 8
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 36390

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.8136
Number of communities: 8
Elapsed time: 0 seconds
[1] 0.9
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 33567

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.8414
Number of communities: 8
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 36390

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.8005
Number of communities: 8
Elapsed time: 0 seconds
[1] 1
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 33567

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.8292
Number of communities: 9
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 36390

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.7881
Number of communities: 9
Elapsed time: 0 seconds
[1] 1.1
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 33567

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.8173
Number of communities: 9
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 36390

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.7771
Number of communities: 10
Elapsed time: 0 seconds
[1] 1.2
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 33567

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.8054
Number of communities: 9
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 36390

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.7661
Number of communities: 10
Elapsed time: 0 seconds
[1] 1.3
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 33567

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.7935
Number of communities: 9
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 36390

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.7551
Number of communities: 10
Elapsed time: 0 seconds
[1] 1.4
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056
Number of edges: 33567

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.7816
Number of communities: 9
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 36390

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7441

Number of communities: 10

Elapsed time: 0 seconds

[1] 1.5

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 33567

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7697

Number of communities: 9

Elapsed time: 0 seconds

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 36390

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7331

Number of communities: 10

Elapsed time: 0 seconds

[1] 1.6

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 33567

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7595

Number of communities: 11

Elapsed time: 0 seconds

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 36390

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7224

Number of communities: 11

Elapsed time: 0 seconds

[1] 1.7

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 33567

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7494

Number of communities: 11

Elapsed time: 0 seconds

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 36390

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7120

Number of communities: 11

Elapsed time: 0 seconds

[1] 1.8

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 33567

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7393

Number of communities: 12

Elapsed time: 0 seconds

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 36390

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7015

Number of communities: 11

Elapsed time: 0 seconds

[1] 1.9

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 33567

Running Louvain algorithm...

Maximum modularity in 10 random starts: 0.7294

Number of communities: 12

Elapsed time: 0 seconds

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 1056

Number of edges: 36390

```
Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.6911
Number of communities: 11
Elapsed time: 0 seconds
[1] 2
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck
```

```
Number of nodes: 1056
Number of edges: 33567
```

```
Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.7195
Number of communities: 12
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck
```

```
Number of nodes: 1056
Number of edges: 36390
```

```
Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.6809
Number of communities: 12
Elapsed time: 0 seconds
```

```
[10]: library(mclust)
library(viridis)

resolutions <- seq(0.5, 2.0, by = 0.1)

ari_matrix <- matrix(data=NA, nrow=length(resolutions),
  ↪ncol=length(resolutions))
colnames(ari_matrix) <- paste0("gene_r",resolutions)
rownames(ari_matrix) <- paste0("TE_r",resolutions)

for(r_gene in resolutions){
  for(r_TE in resolutions){
    ari <- adjustedRandIndex(clustersResList[[as.character(r_gene)]][,1],
      clustersResList[[as.character(r_TE)]][,2])
    ari_matrix[paste0("TE_r",r_TE),paste0("gene_r",r_gene)] <- ari
  }
}
ari_matrix

p <- pheatmap(ari_matrix, display_numbers = T, color = mako(100, alpha = 1,
  ↪begin = 0, end = 1, direction = 1)[],
  border_color = NA, number_format = "%.3f", number_color="black",
```

```

cluster_rows = FALSE, cluster_columns = FALSE,
cellwidth = 25, cellheight = 25)

pdf(paste0("figures_", dataset_id, "/ARI_celltypes_SoloTE_noplatelet.pdf"),
    width=9.5, height=9)
p
dev.off()
# ari_values <- sapply(resolutions, function(r) {
#   adjustedRandIndex(clustersResList[[as.character(r)]][,1],
#   clustersResList[[as.character(r)]][,2])
# })

# plot(resolutions, ari_values, type = "b",
#   xlab = "Resolution", ylab = "ARI",
#   main = "Gene vs Transposon Clustering Similarity")

```

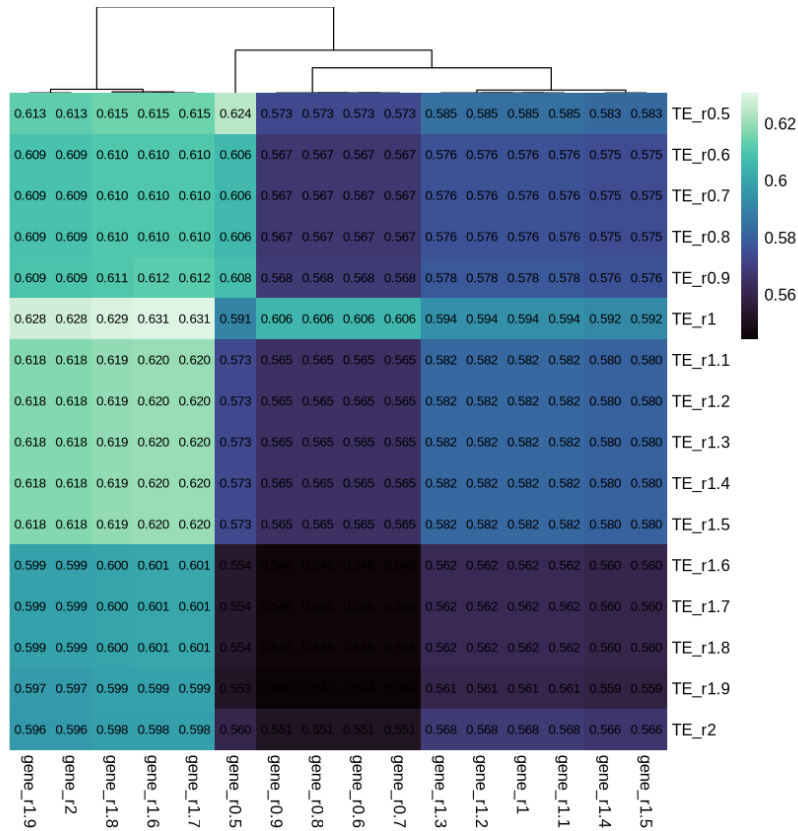
Package 'mclust' version 6.1.1

Type 'citation("mclust")' for citing this R package in publications.

Loading required package: viridisLite

	gene_r0.5	gene_r0.6	gene_r0.7	gene_r0.8	gene_r0.9	gene_r1
TE_r0.5	0.6242549	0.5732102	0.5732102	0.5732102	0.5732102	0.5845955
TE_r0.6	0.6060824	0.5669824	0.5669824	0.5669824	0.5669824	0.5761079
TE_r0.7	0.6060824	0.5669824	0.5669824	0.5669824	0.5669824	0.5761079
TE_r0.8	0.6060824	0.5669824	0.5669824	0.5669824	0.5669824	0.5761079
TE_r0.9	0.6075431	0.5684287	0.5684287	0.5684287	0.5684287	0.5775902
TE_r1	0.5909510	0.6056287	0.6056287	0.6056287	0.6056287	0.5938807
TE_r1.1	0.5732854	0.5650581	0.5650581	0.5650581	0.5650581	0.5817192
TE_r1.2	0.5732854	0.5650581	0.5650581	0.5650581	0.5650581	0.5817192
TE_r1.3	0.5732854	0.5650581	0.5650581	0.5650581	0.5650581	0.5817192
TE_r1.4	0.5732854	0.5650581	0.5650581	0.5650581	0.5650581	0.5817192
TE_r1.5	0.5732854	0.5650581	0.5650581	0.5650581	0.5650581	0.5817192
TE_r1.6	0.5544214	0.5458352	0.5458352	0.5458352	0.5458352	0.5621403
TE_r1.7	0.5544214	0.5458352	0.5458352	0.5458352	0.5458352	0.5621403
TE_r1.8	0.5544214	0.5458352	0.5458352	0.5458352	0.5458352	0.5621403
TE_r1.9	0.5529712	0.5443478	0.5443478	0.5443478	0.5443478	0.5606168
TE_r2	0.5601100	0.5513482	0.5513482	0.5513482	0.5513482	0.5679792

pdf: 2



```
[11]: TEclusters <- clustersResList[["1"]][,2]
      table(clustersResList[["1"]][,2])

      GENEclusters <- clustersResList[["1.7"]][,1]
      table(clustersResList[["1.7"]][,1])
```

```

  1  2  3  4  5  6  7  8  9
155 148 142 140 132 110 104 90 35

  1  2  3  4  5  6  7  8  9 10 11
142 131 124 112 102 97 95 89 76 47 41
```

```
[12]: options(repr.plot.width=8, repr.plot.height=7)

concordanceTable <- table(TEclusters, GENEclusters)

rownames(concordanceTable) <- paste0("TE_cluster", rownames(concordanceTable))
colnames(concordanceTable) <- paste0("GENE_cluster", colnames(concordanceTable))
```

```
# pheatmap(concordanceTable, display_numbers = T, main = "Cluster concordance - TEs VS Genes",
#           fontsize_number = 12, fontsize = 12, annotation_names_row = TRUE,
#           color = alpha(brewer.pal(9, 'Purples')[1:7], 0.5),
#           cluster_rows=FALSE, cluster_cols=FALSE,
#           border_color = NA, number_format = "%.0f", cellwidth = 30,
#           cellheight = 30)
```

```
[13]: library(clue)
library(grid)
options(repr.plot.width=9, repr.plot.height=8)

# Suppose your matrix has more rows than columns
nr <- nrow(concordanceTable)
nc <- ncol(concordanceTable)

if(nr > nc){
  # pad with zeros to make it square
  M <- cbind(concordanceTable, matrix(0, nrow = nr, ncol = nr - nc))
} else {
  M <- rbind(concordanceTable, matrix(0, nrow = nc - nr, ncol = nc))
}

perm <- solve_LSAP(M, maximum = TRUE)

# keep only the original columns
M_reordered <- M[, perm[1:nc]]
rownames(M_reordered)

ph <- pheatmap( M_reordered[(rownames(M_reordered)!=""),!is.na(colnames(M_reordered))] ,
               display_numbers = T, color = colorRampPalette(brewer.pal(9, 'Blues'))[1:
               7])(100), #adjustcolor((colorRampPalette(carto_pal(name="Emrld")))(100), alpha.
               f=1),
               main = "Cluster concordance - TEs VS Genes",
               cluster_rows=FALSE, cluster_cols=FALSE,
               fontsize_number = 12, fontsize = 12, annotation_names_row =
               TRUE, number_color = "black",
               border_color = NA, number_format = "%.0f", cellwidth = 30, cellheight
               = 30)

# write to PDF safely
```

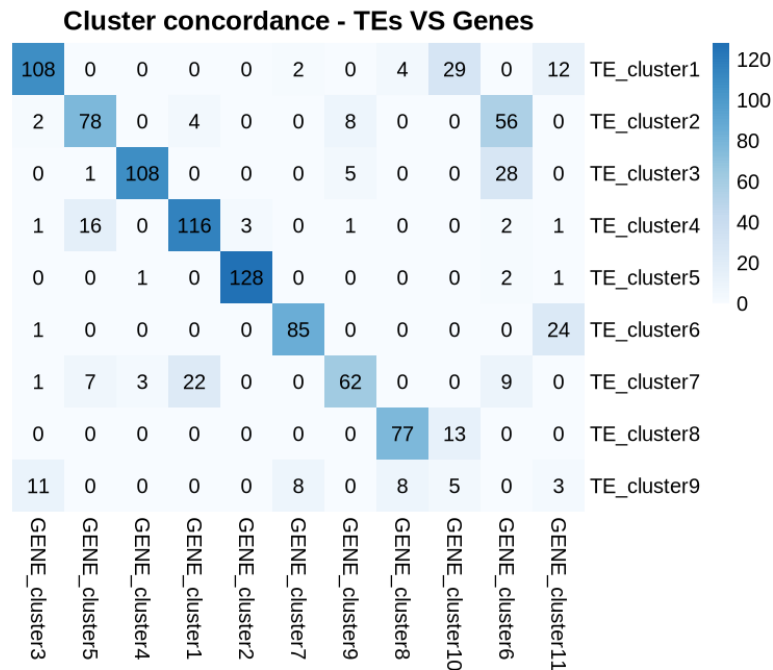
```
pdf(paste0("figures_", dataset_id, "/"
  ↳heatmap_cluster_clusters_concordance_SoloTE_noplatelet.pdf"), width=8,
  ↳height=9)

grid::grid.newpage()
grid::grid.draw(ph$gtable)

dev.off()
```

1. 'TE_cluster1' 2. 'TE_cluster2' 3. 'TE_cluster3' 4. 'TE_cluster4' 5. 'TE_cluster5'
6. 'TE_cluster6' 7. 'TE_cluster7' 8. 'TE_cluster8' 9. 'TE_cluster9' 10. " 11. "

pdf: 2



```
[14]: options(repr.plot.width=8, repr.plot.height=8)

concordanceTable <- table(TEclusters, GENEclusters)
```

```

rownames(concordanceTable) <- paste0("TE_cluster", rownames(concordanceTable))
colnames(concordanceTable) <- paste0("GENE_cluster", colnames(concordanceTable))

concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

# Suppose your matrix has more rows than columns
nr <- nrow(concordanceTable)
nc <- ncol(concordanceTable)
concordanceTable
if(nr > nc){
  # pad with zeros to make it square
  M <- cbind(concordanceTable, matrix(0, nrow = nr, ncol = nr - nc))
} else {
  M <- concordanceTable
}
perm <- solve_LSAP(M, maximum = TRUE)
M
perm
# keep only the original columns
M_reordered <- M[, perm]
M_reordered <- M_reordered[, colSums(M_reordered)>0]

ph <- pheatmap(M_reordered,
  display_numbers = TRUE, color = colorRampPalette(brewer.
    ↪pal(9, 'Blues'))[1:
    ↪7])(100), #adjustcolor((colorRampPalette(carto_pal(name="Emrld")))(100), alpha.
    ↪f=1),
  main = "TE cluster - cell type concordance",
  cluster_rows=FALSE, cluster_cols=FALSE,
  fontsize_number = 12, fontsize = 12, annotation_names_row =
    ↪TRUE, number_color = "black",
  border_color = NA, number_format = "%.0f", cellwidth = 30, cellheight
    ↪= 30)
dev.off()

pdf(paste0("figures_", dataset_id, "/"
  ↪heatmap_cluster_celltype_concordance_SoloTE_noPlatelet.pdf"), width=8,
  ↪height=8)
grid::grid.newpage()
grid::grid.draw(ph$gtable)
dev.off()

```

	B	Dendritic	Monocytes_CD14pos	Monocytes_FCGR3Apos	NK	T_CD4_Memory
0	0	14	112	29	0	0
1	0	0	2	0	1	83
2	0	0	0	0	124	1
3	2	0	2	0	2	21
4	130	0	1	0	1	0
5	0	108	2	0	0	0
6	0	0	1	0	4	27
7	0	0	0	90	0	0
8	0	10	12	13	0	0

	T_CD4_Naive	T_CD8
0	0	0
1	1	61
2	0	17
3	110	3
4	0	0
5	0	0
6	21	51
7	0	0
8	0	0

A matrix: 9×9 of type dbl

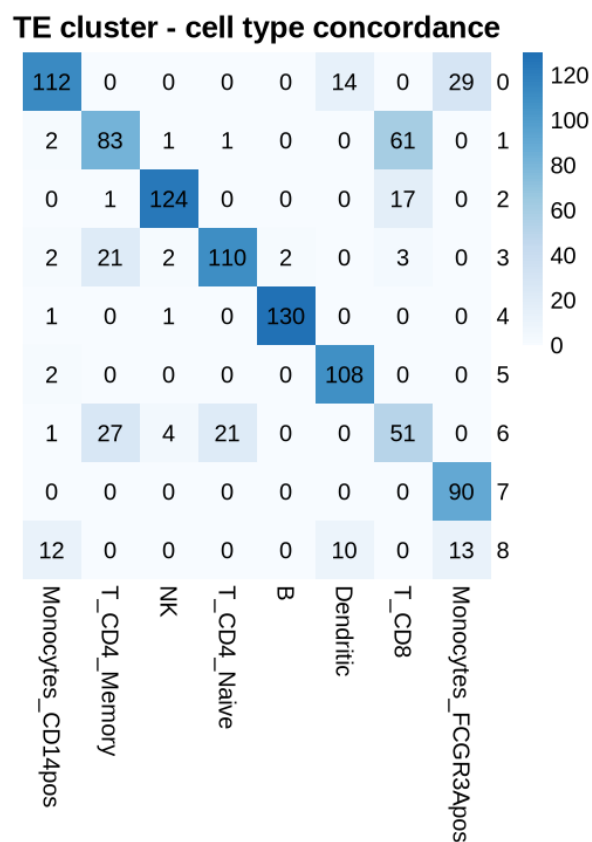
	B	Dendritic	Monocytes_CD14pos	Monocytes_FCGR3Apos	NK	T_CD4_Memory
0	0	14	112	29	0	0
1	0	0	2	0	1	83
2	0	0	0	0	124	1
3	2	0	2	0	2	21
4	130	0	1	0	1	0
5	0	108	2	0	0	0
6	0	0	1	0	4	27
7	0	0	0	90	0	0
8	0	10	12	13	0	0

Optimal assignment:

1 => 3, 2 => 6, 3 => 5, 4 => 7, 5 => 1, 6 => 2, 7 => 8, 8 => 4, 9 => 9

null device: 1

png: 2



```
[15]: options(repr.plot.width=10, repr.plot.height=8)

concordanceTable <- table(TEclusters, GENEclusters)

rownames(concordanceTable) <- paste0("TE_cluster", rownames(concordanceTable))
colnames(concordanceTable) <- paste0("GENE_cluster", colnames(concordanceTable))

concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

# Suppose your matrix has more rows than columns
nr <- nrow(concordanceTable)
nc <- ncol(concordanceTable)
concordanceTable
```

```

if(nr > nc){
  # pad with zeros to make it square
  M <- cbind(concordanceTable, matrix(0, nrow = nr, ncol = nr - nc))
} else {
  M <- concordanceTable
}
perm <- solve_LSAP(M, maximum = TRUE)
M
perm
# keep only the original columns
M_reordered <- M[, perm]
M_reordered <- M_reordered[, colSums(M_reordered)>0]

ph <- pheatmap(t(M_reordered),
  display_numbers = TRUE, color = colorRampPalette(brewer.
    ↪pal(9, 'Blues')[1:
    ↪7])(100), #adjustcolor((colorRampPalette(carto_pal(name="Emrld")))(100), alpha.
    ↪f=1),
  main = "TE cluster - cell type concordance",
  cluster_rows=TRUE, cluster_cols=TRUE,
  fontsize_number = 12, fontsize = 12, annotation_names_row =
    ↪TRUE, number_color = "black",
  border_color = NA, number_format = "%.0f", cellwidth = 30, cellheight
    ↪= 30)

pdf(paste0("figures_", dataset_id, "/"
  ↪heatmap_cluster_celltype_concordance_SoloTE_clustered_noPlatelet.pdf"),
  ↪width=8, height=8)
grid::grid.newpage()
grid::grid.draw(ph$gtable)
dev.off()

ph <- pheatmap(t(M_reordered),
  display_numbers = TRUE, color = colorRampPalette(brewer.
    ↪pal(9, 'Blues')[1:
    ↪7])(100), #adjustcolor((colorRampPalette(carto_pal(name="Emrld")))(100), alpha.
    ↪f=1),
  main = "TE cluster - cell type concordance",
  cluster_rows=F, cluster_cols=F,
  fontsize_number = 12, fontsize = 14, annotation_names_row =
    ↪TRUE, number_color = "black",
  angle_col=0,
  border_color = NA, number_format = "%.0f", cellwidth = 30, cellheight
    ↪= 30)

```

```
pdf(paste0("figures_", dataset_id, "/"
  ↳heatmap_cluster_celltype_concordance_SoloTE_horizontal_noPlatelet.pdf"),
  ↳width=10, height=8)
grid::grid.newpage()
grid::grid.draw(ph$htable)
dev.off()

rownames(M_reordered) <- paste0("TE cluster ", rownames(M_reordered))

ph <- pheatmap(t(M_reordered),
  display_numbers = TRUE, color = colorRampPalette(brewer.
  ↳pal(9, 'Blues')[1:
  ↳7])(100),#adjustcolor((colorRampPalette(carto_pal(name="Emrld")))(100),alpha.
  ↳f=1),
  main = "TE cluster - cell type concordance",
  cluster_rows=F, cluster_cols=F,
  fontsize_number = 12, fontsize = 14, annotation_names_row =
  ↳TRUE, number_color = "black",
  angle_col=45,
  border_color = NA, number_format = "%.0f", cellwidth = 30, cellheight=
  ↳= 30)

pdf(paste0("figures_", dataset_id, "/"
  ↳heatmap_cluster_celltype_concordance_SoloTE_horizontal_longClusterNames.
  ↳pdf"), width=10, height=8)
grid::grid.newpage()
grid::grid.draw(ph$htable)
dev.off()
```

	B	Dendritic	Monocytes_CD14pos	Monocytes_FCGR3Apos	NK	T_CD4_Memory
0	0	14	112	29	0	0
1	0	0	2	0	1	83
2	0	0	0	0	124	1
3	2	0	2	0	2	21
4	130	0	1	0	1	0
5	0	108	2	0	0	0
6	0	0	1	0	4	27
7	0	0	0	90	0	0
8	0	10	12	13	0	0

	T_CD4_Naive	T_CD8
0	0	0
1	1	61
2	0	17
3	110	3

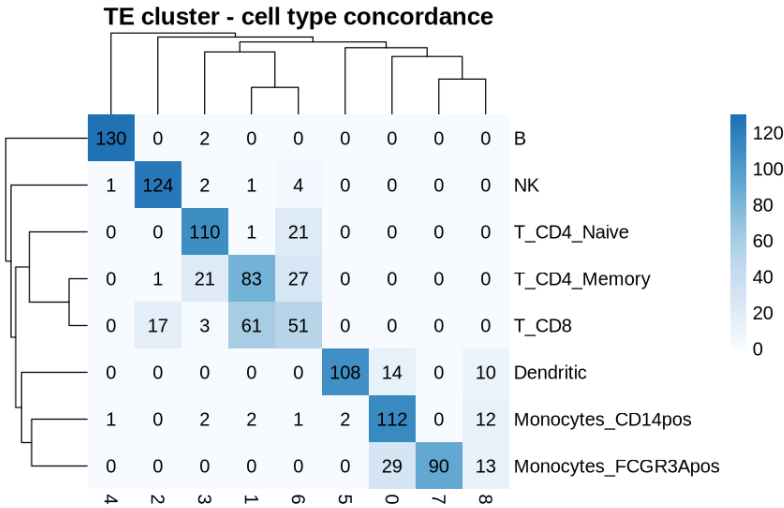
4	0	0
5	0	0
6	21	51
7	0	0
8	0	0

A matrix: 9 × 9 of type dbl

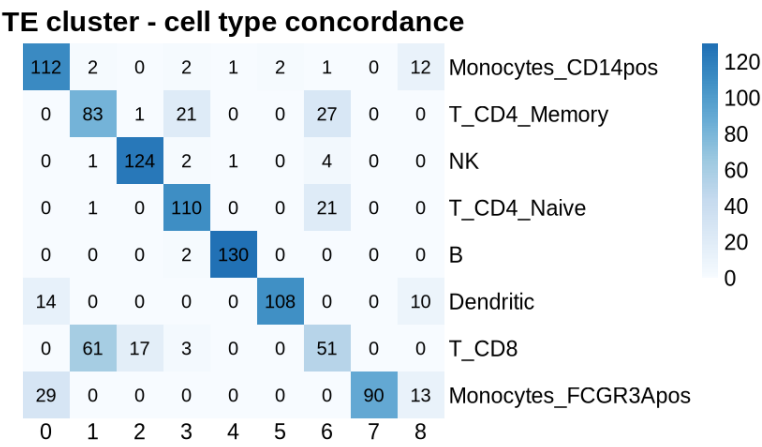
	B	Dendritic	Monocytes_CD14pos	Monocytes_FCGR3Apos	NK	T_CD4_Naive
0	0	14	112	29	0	0
1	0	0	2	0	1	83
2	0	0	0	0	124	1
3	2	0	2	0	2	21
4	130	0	1	0	1	0
5	0	108	2	0	0	0
6	0	0	1	0	4	27
7	0	0	0	90	0	0
8	0	10	12	13	0	0

Optimal assignment:
1 => 3, 2 => 6, 3 => 5, 4 => 7, 5 => 1, 6 => 2, 7 => 8, 8 => 4, 9 => 9

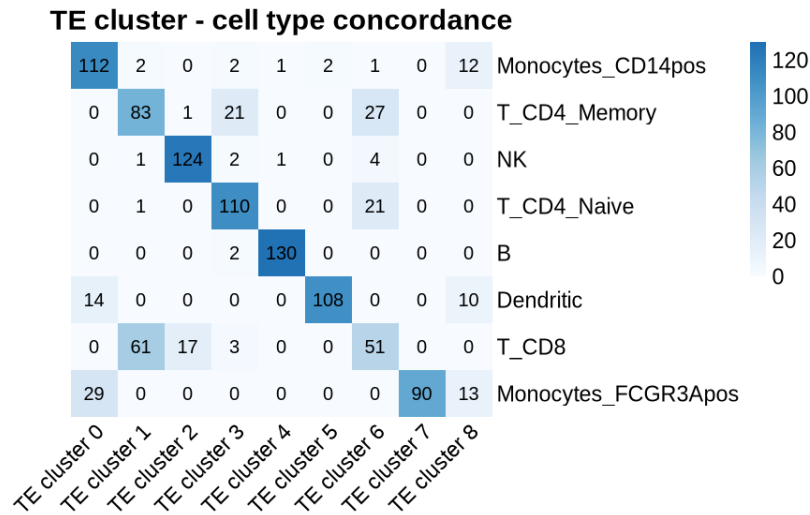
pdf: 2



pdf: 2



pdf: 2



```
[16]: options(repr.plot.width=10, repr.plot.height=8)

concordanceTable <- table(TEclusters, GENEclusters)

rownames(concordanceTable) <- paste0("TE_cluster", rownames(concordanceTable))
colnames(concordanceTable) <- paste0("GENE_cluster", colnames(concordanceTable))

concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

# Suppose your matrix has more rows than columns
nr <- nrow(concordanceTable)
nc <- ncol(concordanceTable)
concordanceTable
if(nr > nc){
  # pad with zeros to make it square
  M <- cbind(concordanceTable, matrix(0, nrow = nr, ncol = nr - nc))
} else {
  M <- concordanceTable
}
```

```

perm <- solve_LSAP(M, maximum = TRUE)
M
perm
# keep only the original columns
M_reordered <- M[, perm]
M_reordered <- M_reordered[, colSums(M_reordered)>0]

ph <- pheatmap(t(M_reordered),
  display_numbers = TRUE, color = colorRampPalette(brewer.
    ↪pal(9, 'Blues')[1:
    ↪7])(100), #adjustcolor((colorRampPalette(carto_pal(name="Emrld")))(100), alpha.
    ↪f=1),
  main = "TE cluster - cell type concordance",
  cluster_rows=TRUE, cluster_cols=TRUE,
    fontsize_number = 12, fontsize = 12, annotation_names_row =
    ↪TRUE, number_color = "black",
    border_color = NA, number_format = "%.0f", cellwidth = 30, cellheight
    ↪= 30)

pdf(paste0("figures_", dataset_id, "/"
  ↪heatmap_cluster_celltype_concordance_SoloTE_clustered_noplatelet.pdf"),
  ↪width=8, height=8)
grid::grid.newpage()
grid::grid.draw(ph$gtable)
dev.off()

rownames(M_reordered) <- paste0("TE cluster ", rownames(M_reordered))

ph <- pheatmap(t(M_reordered),
  display_numbers = TRUE, color = colorRampPalette(brewer.
    ↪pal(9, 'Blues')[1:
    ↪7])(100), #adjustcolor((colorRampPalette(carto_pal(name="Emrld")))(100), alpha.
    ↪f=1),
  main = "TE cluster - cell type concordance",
  cluster_rows=F, cluster_cols=F,
    fontsize_number = 12, fontsize = 14, annotation_names_row =
    ↪TRUE, number_color = "black", angle_col=45,
    border_color = NA, number_format = "%.0f", cellwidth = 30, cellheight
    ↪= 30)

```

```
pdf(paste0("figures_", dataset_id, "/"
  ↳heatmap_cluster_celltype_concordance_SoloTE_horizontal_noplatelet.pdf"),
  ↳width=10, height=8)
grid::grid.newpage()
grid::grid.draw(ph$gttable)
dev.off()
```

	B	Dendritic	Monocytes_CD14pos	Monocytes_FCGR3Apos	NK	T_CD4_Memory
0	0	14	112	29	0	0
1	0	0	2	0	1	83
2	0	0	0	0	124	1
3	2	0	2	0	2	21
4	130	0	1	0	1	0
5	0	108	2	0	0	0
6	0	0	1	0	4	27
7	0	0	0	90	0	0
8	0	10	12	13	0	0

	T_CD4_Naive	T_CD8
0	0	0
1	1	61
2	0	17
3	110	3
4	0	0
5	0	0
6	21	51
7	0	0
8	0	0

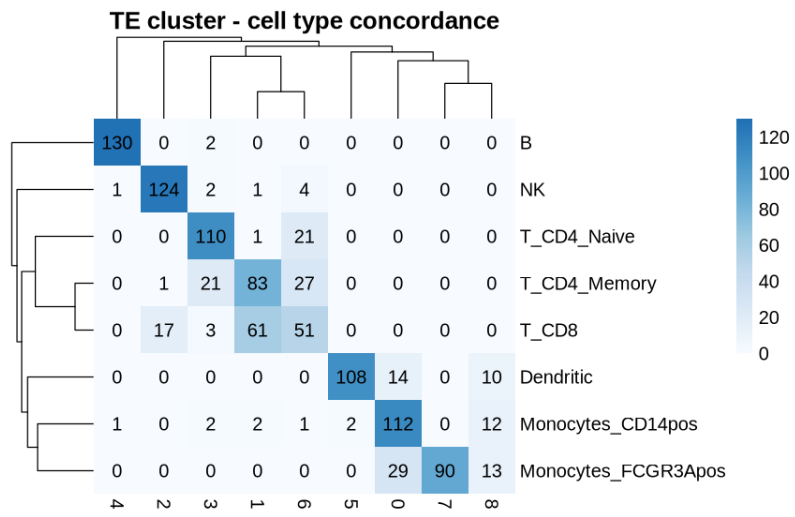
A matrix: 9 × 9 of type dbl

	B	Dendritic	Monocytes_CD14pos	Monocytes_FCGR3Apos	NK	T_CD8
0	0	14	112	29	0	0
1	0	0	2	0	1	83
2	0	0	0	0	124	1
3	2	0	2	0	2	21
4	130	0	1	0	1	0
5	0	108	2	0	0	0
6	0	0	1	0	4	27
7	0	0	0	90	0	0
8	0	10	12	13	0	0

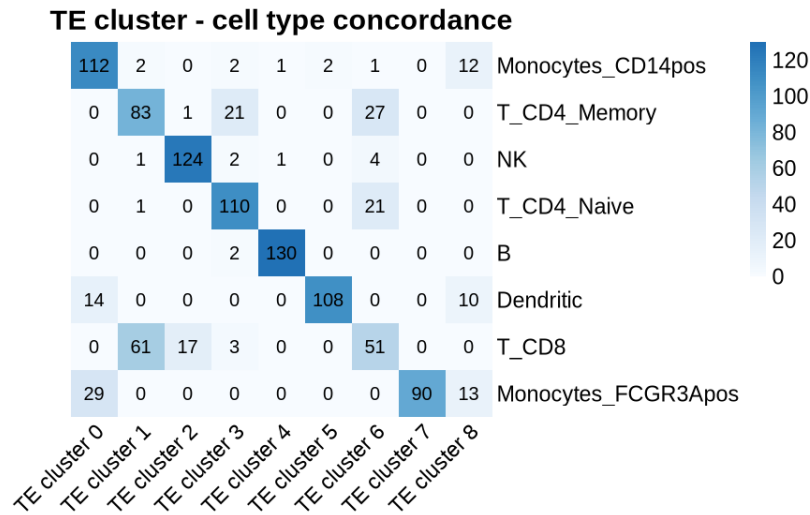
Optimal assignment:

1 => 3, 2 => 6, 3 => 5, 4 => 7, 5 => 1, 6 => 2, 7 => 8, 8 => 4, 9 => 9

pdf: 2



pdf: 2



```
[17]: options(repr.plot.width=10, repr.plot.height=5)
# Comparison to genes

objGenes_SoloTE$celltype_num <- objGenes_SoloTE$seurat_clusters
levels(objGenes_SoloTE$celltype_num) <- 1:
  ↪ length(levels(objGenes_SoloTE$seurat_clusters))

# Add cluster information from objTE_clustered to objGenes_clustered
objGenes_SoloTE$clusters.0.1 <- GENEclusters
objGenes_SoloTE$clusters.0.2 <- TEclusters
objGenes_SoloTE$clusters.0.3 <- NULL
objGenes_SoloTE$clusters.0.4 <- NULL

# Generate the clustree plot
clustree(objGenes_SoloTE, prefix = "clusters.", node_text_size=4,
  ↪ node_text_angle=45) +
  theme(text=element_text(size=15)) +
  scale_color_manual(values=alpha(c("#f9b294", "#C87D95"), 0.5)) +
  scale_size(range = c(3,20)) +
```

```

    guides(colour = FALSE)
ggsave(paste0("figures_", dataset_id, "/clustree_clusters.pdf"), width=10,
  height=7 )

objTE_SoloTE$RNAclusters <- objGenes_SoloTE$seurat_clusters

options(repr.plot.width=15, repr.plot.height=9)

# Add cluster information from objTE_clustered to objGenes_clustered
objGenes_SoloTE$clusters.0.1 <- objGenes_SoloTE$celltype
objGenes_SoloTE$clusters.0.2 <- GENEclusters
objGenes_SoloTE$clusters.0.3 <- TEclusters
objGenes_SoloTE$clusters.0.4 <- objGenes_SoloTE$celltype

concordanceTable <- table(objGenes_SoloTE$seurat_clusters,
  objTE_SoloTE$seurat_clusters)

pheatmap(concordanceTable, display_numbers = T, color = brewer.pal(9, 'Blues')[1:
  6],
  border_color = NA, number_format = "%.0f", cellwidth = 25, cellheight=
  25)

set.seed(123)
# Generate the clustree plot
clustree(objGenes_SoloTE, prefix = "clusters.", node_text_size=4,
  node_text_angle=45) +
  theme(text=element_text(size=15)) +
  scale_color_manual(values=alpha(c("grey90", "#f9b294", "#C87D95", "grey90"), 0.
  5)) +
  scale_size(range = c(3,20)) +
  guides(colour = FALSE)
ggsave(paste0("figures_", dataset_id, "/clustree_clusters_allcelltypes_noplatelet.
  pdf"), width=10, height=7 )

objTE_SoloTE$RNAclusters <- objGenes_SoloTE$seurat_clusters

```

Warning message:

"Cannot find cell-level meta data named clusters.0.3"

Warning message:

"Cannot find cell-level meta data named clusters.0.4"

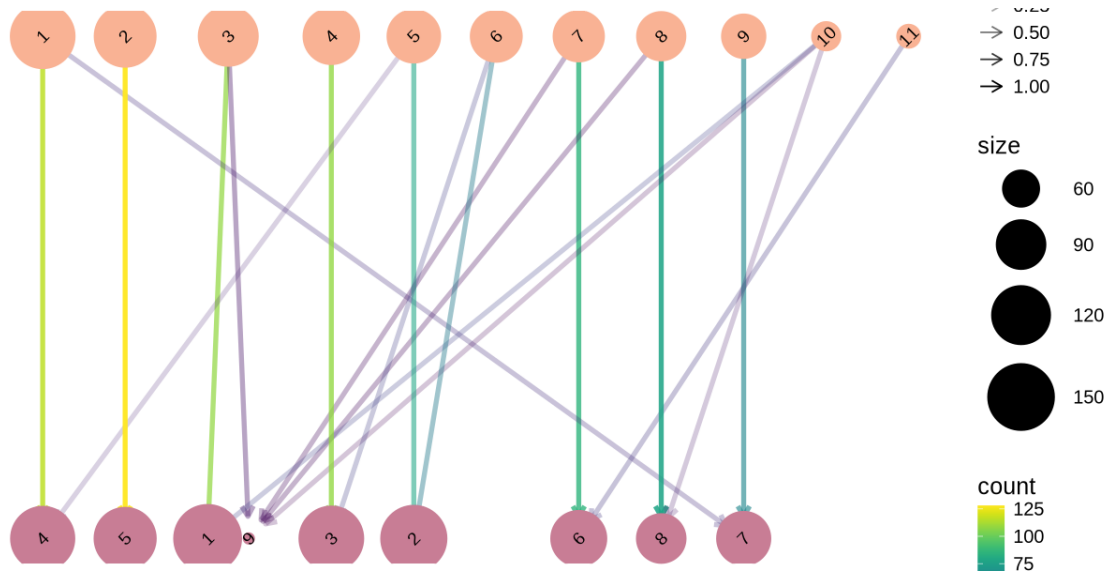
Scale for **size** is already present.

Adding another scale for **size**, which will replace the existing scale.

Warning message:

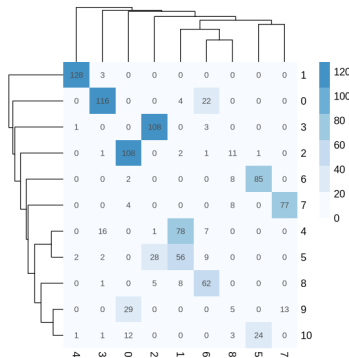
"The `<scale>` argument of `guides()` cannot be `FALSE`. Use "none" instead as

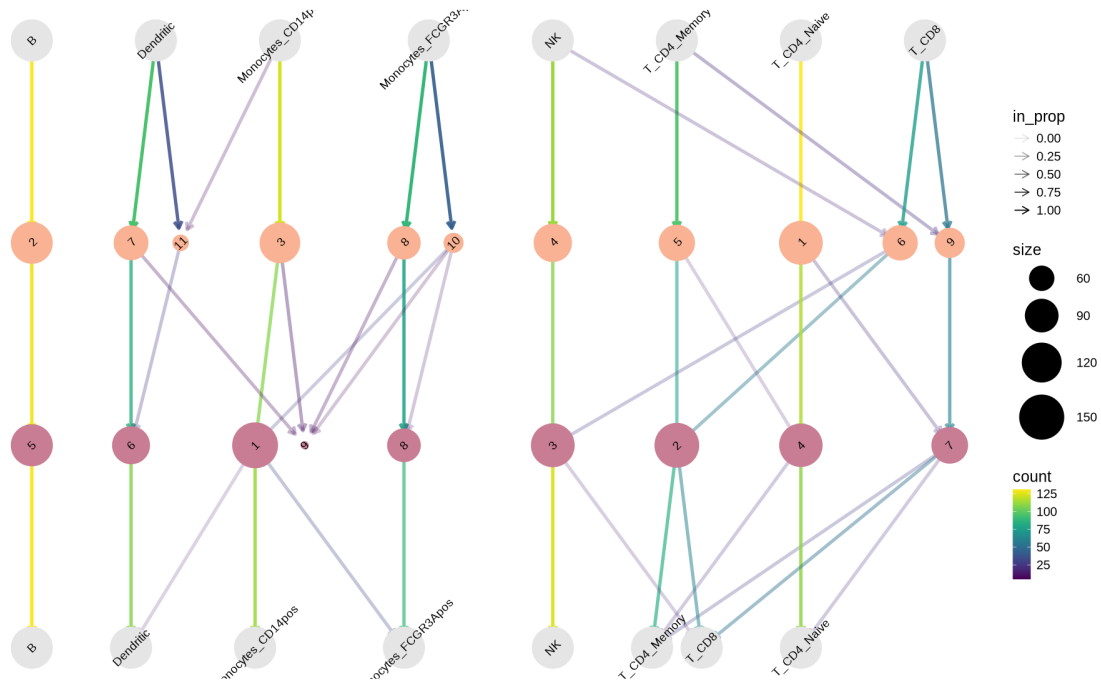
of ggplot2 3.3.4."



Scale for `size` is already present.

Adding another scale for `size`, which will replace the existing scale.





```
[85]: PBMCcelltypeColors <- c("B"="#6D5A5D",
                              "Monocytes_CD14pos"="#4E876D",
                              "Dendritic"="#C4B3AB", "#E78063", "#c492a7",
                              "Monocytes_FCGR3Apos"="#81B29A",
                              # "Platelet"="#c4b3ab",
                              "T_CD4_Memory"="#84B6D6",
                              "T_CD4_Naive"="#6790B5",
                              "T_CD8"="#3D405B",
                              "NK"= "#F2CC8F")
```

```
[86]: options(repr.plot.width=8, repr.plot.height=3)
concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

df <- melt(concordanceTable)
df <- df[df$value!=0,]
colnames(df) <- c("cluster", "celltype", "nCells")
df$cluster <- as.character(df$cluster)
df$clusterSize <- table(objTE_SoloTE$seurat_clusters)[df$cluster]
df$percentage <- as.numeric(df$nCells / df$clusterSize *100)

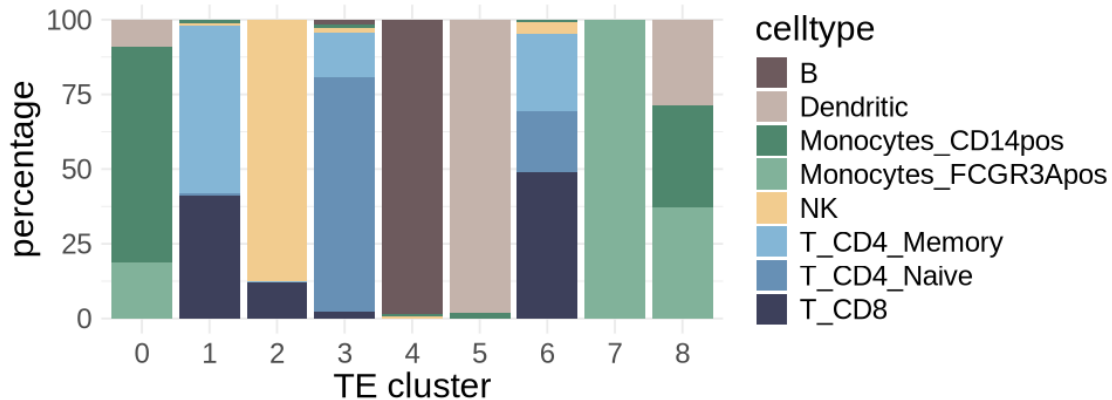
df$cluster <- factor(df$cluster, levels=c(0,1:length(unique(df$cluster))))

ggplot(df, aes(x=cluster, y=percentage, fill=celltype)) +
  geom_col() +
```

```

xlab("TE cluster") +
scale_fill_manual(values=PBMCelltypeColors)+
theme_minimal() + theme(text=element_text(size=18))
ggsave("figures_10xPBM/clusterIdentity_barplot_SoloTE_noplatelet.pdf",
width=8, height=3)

```



```

[123]: options(repr.plot.width=8, repr.plot.height=3)
concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

df <- melt(concordanceTable)
df <- df[df$value!=0,]
colnames(df) <- c("cluster", "celltype", "nCells")
df$cluster <- as.character(df$cluster)
df$clusterSize <- table(objTE_SoloTE$seurat_clusters)[df$cluster]
df$percentage <- as.numeric(df$nCells / df$clusterSize *100)

df <- df %>%
  group_by(cluster) %>%
  mutate(main = celltype[which.max(percentage)]) %>%
  ungroup() %>%
  arrange(main, desc(percentage))
df$cluster <- factor(df$cluster, levels = unique(df$cluster))

ggplot(df, aes(x=cluster, y=percentage, fill=celltype)) +
  geom_col() +
  xlab("TE cluster") +
  # geom_text(data = sizes,
  #           aes(x = factor(cluster), y = 105, label = clusterSize),
  #           inherit.aes = FALSE) +
  scale_fill_manual(values=PBMCelltypeColors)+
  theme_minimal() + theme(text=element_text(size=18))

```

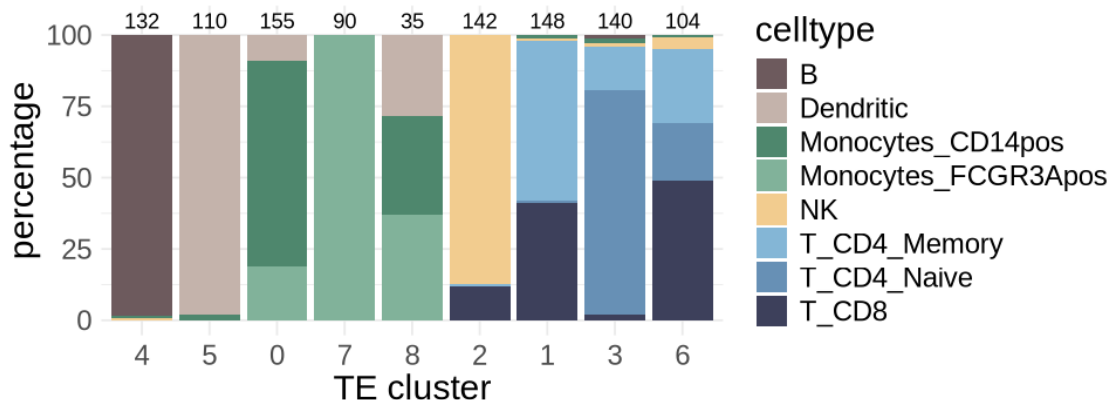
```
ggsave("figures_10xPBM/clusterIdentity_perc_barplot_SoloTE_noplatelet.pdf",
       width=8, height=3)
```

Don't know how to automatically pick scale for object of type
<table>.

Defaulting to continuous.

Don't know how to automatically pick scale for object of type
<table>.

Defaulting to continuous.



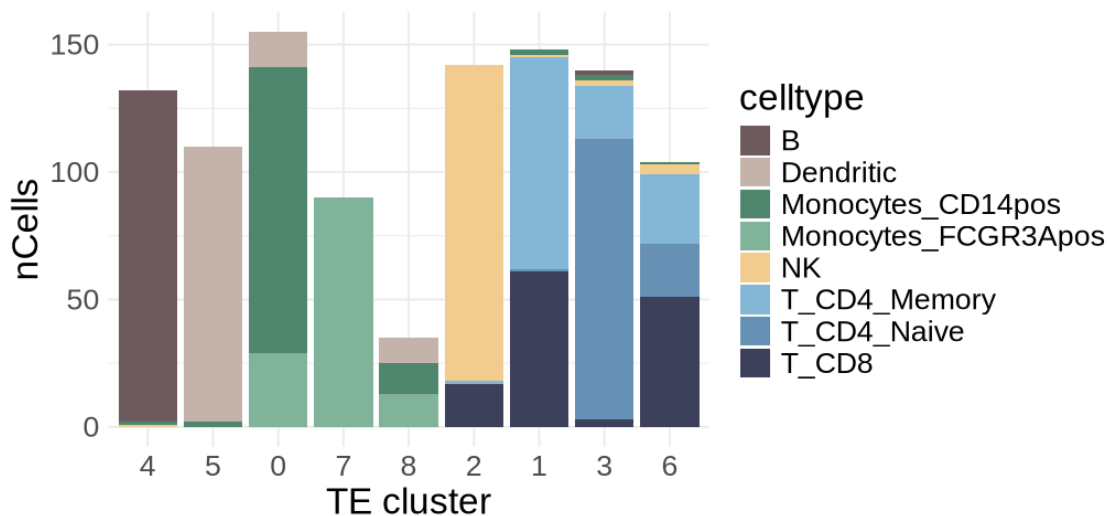
```
[128]: options(repr.plot.width=8, repr.plot.height=3)
concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

df <- melt(concordanceTable)
df <- df[df$value!=0,]
colnames(df) <- c("cluster", "celltype", "nCells")
df$cluster <- as.character(df$cluster)
df$clusterSize <- table(objTE_SoloTE$seurat_clusters)[df$cluster]
df$percentage <- as.numeric(df$nCells / df$clusterSize *100)

df <- df %>%
  group_by(cluster) %>%
  mutate(main = celltype[which.max(percentage)]) %>%
  ungroup() %>%
  arrange(main, desc(percentage))
df$cluster <- factor(df$cluster, levels = unique(df$cluster))

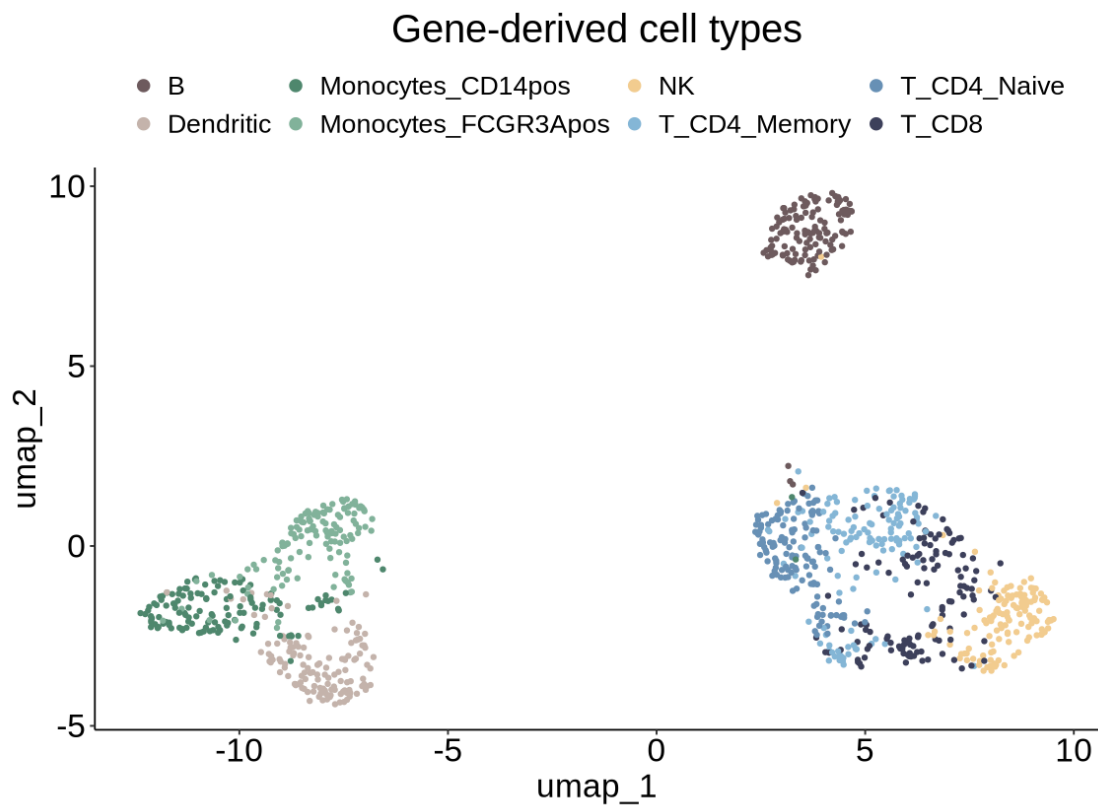
ggplot(df, aes(x=cluster, y=nCells, fill=celltype)) +
  geom_col() +
  xlab("TE cluster") +
  # geom_text(data = sizes,
```

```
#       aes(x = factor(cluster), y = 105, label = clusterSize),
#       inherit.aes = FALSE) +
scale_fill_manual(values=PBMCcelltypeColors)+
theme_minimal() + theme(text=element_text(size=18))
ggsave(paste0("figures_", dataset_id, "/"
↪clusterIdentity_barplot_SoloTE_noplatelet.pdf"), width=8, height=3)
```



```
[130]: options(repr.plot.width=9.5, repr.plot.height=7)

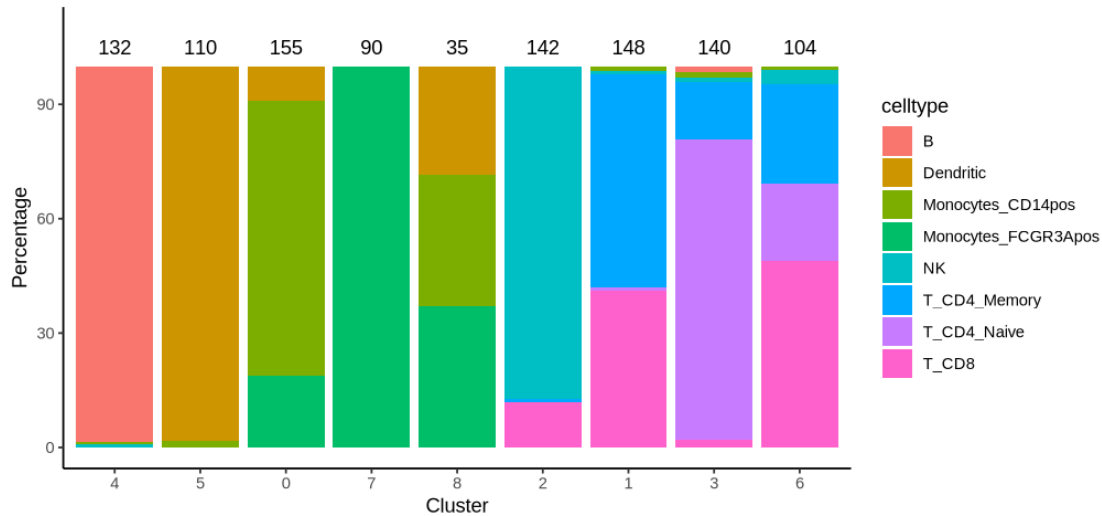
DimPlot(objTE_SoloTE, reduction = "umap", group.by = "celltype",
        cols=PBMCcelltypeColors,
        shuffle=T, pt.size = 1) +
  ggtitle("Gene-derived cell types") +
  theme_pubr() +
  theme(text=element_text(size=20), plot.title = element_text(hjust=0.5))
ggsave(paste0("figures_", dataset_id, "/"
↪umap_TEs_locus_celltypes_SoloTE_noplatelet.pdf"), device = "pdf", width=9.5,
↪height=7)
```



```
[129]: sizes <- df %>% distinct(cluster, clusterSize)

ggplot(df, aes(factor(cluster), percentage, fill = celltype)) +
  geom_col() +
  geom_text(data = sizes,
            aes(x = factor(cluster), y = 105, label = clusterSize),
            inherit.aes = FALSE) +
  scale_y_continuous(limits = c(0, 110)) +
  theme_classic() +
  labs(y = "Percentage", x = "Cluster")
```

Don't know how to automatically pick scale for object of type
 <table>.
 Defaulting to continuous.



```
[ ]: options(repr.plot.width=8, repr.plot.height=3)
concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

df <- melt(concordanceTable)
df <- df[df$value!=0,]
colnames(df) <- c("cluster", "celltype", "nCells")
df$celltypeSize <- table(objTE_SoloTE$celltype)[df$celltype]
df$percentage <- as.numeric(df$nCells / df$clusterSize *100)

ggplot(df, aes(x=cluster, y=percentage, fill=celltype)) +
  geom_col() +
  xlab("TE cluster") +
  scale_fill_manual(values=PBMCelltypeColors)+
  theme_minimal() + theme(text=element_text(size=18))
ggsave("figures_10xPBM/clusterIdentity_barplot_SoloTE_noplatelet.pdf",
  width=8, height=3)
```

```
[ ]: # some way to show the number of cells in each cluster? height proportional to
  size? Order by size? N on top?
```

```
[68]: PBMCelltypeColors <- c("B"="#5e50a1",
  "Monocytes_CD14pos"="#fcbe60",
  "Dendritic"="#3787bc",
  "Monocytes_FCGR3Apos"="#f39b5c",
  "T_CD4_Naive"="#aadca2",
  "T_CD4_Memory"="#67c1a3",
  "T_CD8"="#4fa4b0",
  "NK"= "#D4404F")
```

```
[66]: , "#f39b5c", "#D4404F", "#5e50a1", "#3787bc", "#aadca2"␣
↪, "#67c1a3", "#4fa4b0", "#FEE9AE", "#ac5d8f", "#67132C")
```

A data.frame: 31 × 5

	cluster <fct>	celltype <fct>	nCells <int>	clusterSize <table[1d]>	percentage <dbl>
10	0	Dendritic	14	155	9.0322581
19	0	Monocytes_CD14pos	112	155	72.2580645
28	0	Monocytes_FCGR3Apos	29	155	18.7096774
20	1	Monocytes_CD14pos	2	148	1.3513514
38	1	NK	1	148	0.6756757
47	1	T_CD4_Memory	83	148	56.0810811
56	1	T_CD4_Naive	1	148	0.6756757
65	1	T_CD8	61	148	41.2162162
39	2	NK	124	142	87.3239437
48	2	T_CD4_Memory	1	142	0.7042254
66	2	T_CD8	17	142	11.9718310
4	3	B	2	140	1.4285714
22	3	Monocytes_CD14pos	2	140	1.4285714
40	3	NK	2	140	1.4285714
49	3	T_CD4_Memory	21	140	15.0000000
58	3	T_CD4_Naive	110	140	78.5714286
67	3	T_CD8	3	140	2.1428571
5	4	B	130	132	98.4848485
23	4	Monocytes_CD14pos	1	132	0.7575758
41	4	NK	1	132	0.7575758
15	5	Dendritic	108	110	98.1818182
24	5	Monocytes_CD14pos	2	110	1.8181818
25	6	Monocytes_CD14pos	1	104	0.9615385
43	6	NK	4	104	3.8461538
52	6	T_CD4_Memory	27	104	25.9615385
61	6	T_CD4_Naive	21	104	20.1923077
70	6	T_CD8	51	104	49.0384615
35	7	Monocytes_FCGR3Apos	90	90	100.0000000
18	8	Dendritic	10	35	28.5714286
27	8	Monocytes_CD14pos	12	35	34.2857143
36	8	Monocytes_FCGR3Apos	13	35	37.1428571

```
[ ]:
```